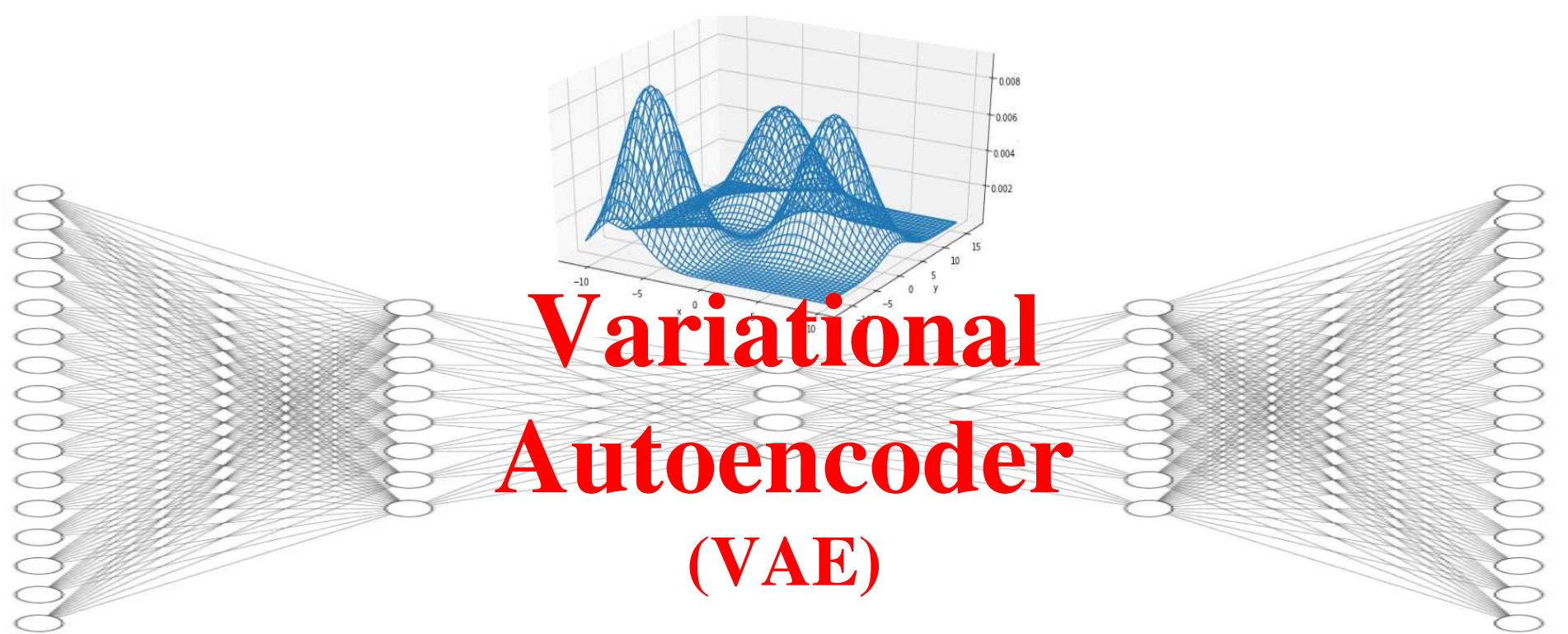
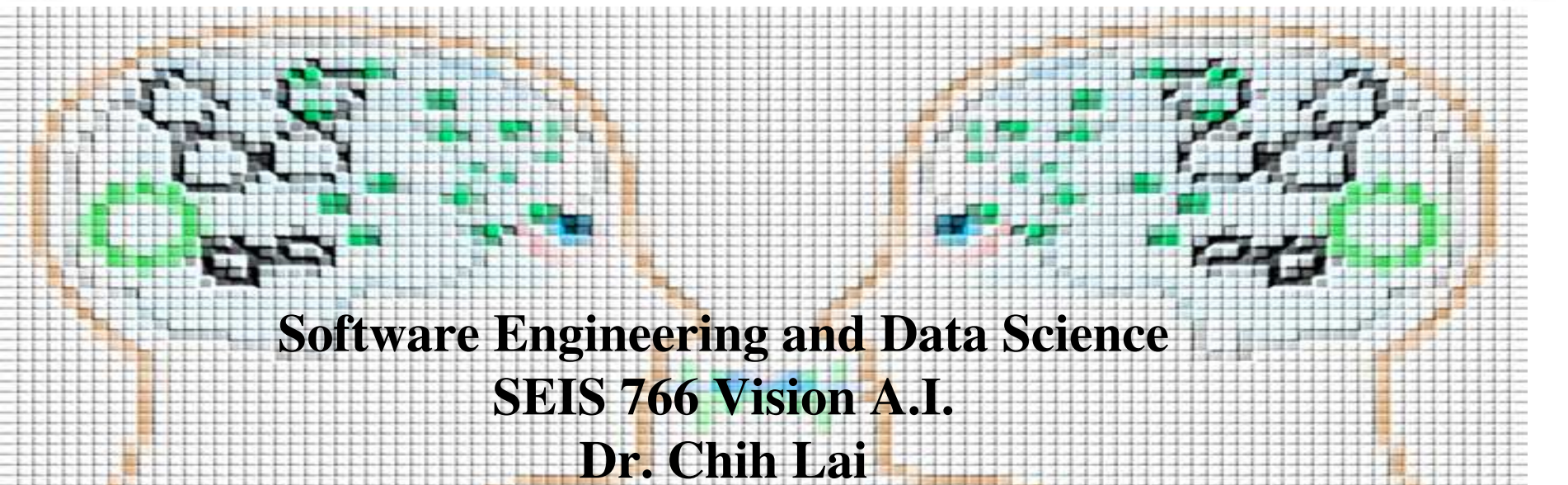




Variational Autoencoder (VAE)



Software Engineering and Data Science
SEIS 766 Vision A.I.
Dr. Chih Lai

766 Topics from Different Angles

- Object Identification & more
- AutoEncoder (AE)
- Variational AE / VAE
- GAN-1: GAN, C-GAN, W-GAN
- GAN-2: Cycle-GAN, UNIT-GAN
- (Stable) Diffusion, Style Xfer, DeepDream
- CLIP / Classification

Grouping by Characteristics

- Object Identification & more
- AutoEncoder (AE), Variational AE (VAE)
- CLIP / Classification
- GAN-2: Cycle-GAN, UNIT-GAN
- Style Xfer
- GAN-1: GAN, C-GAN, W-GAN
- (Stable) Diffusion, DeepDream

Applicability

- Basic image processing, colors

Discussion Order

- Object Identification & more
- AutoEncoder (AE)
- Variational AE / VAE
- **CLIP** (**C**ontrastive **L**anguage-**I**mage **P**re-training)
- GAN + **Quality** measures
- Cycle-GAN, UNIT-GAN
- C-GAN, W-GAN
- Style Xfer
- (Stable) Diffusion, DeepDream
- DNN for Image Processing
- Morphological Ops in traditional Image Processing



VAE References

■ Train Variational Autoencoder (VAE) to Generate Images

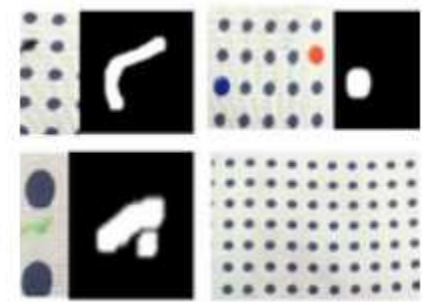
- <https://www.mathworks.com/help/deeplearning/ug/train-a-variational-autoencoder-vae-to-generate-images.html>

■ Generate Digit Images Using Variational Autoencoder on Intel CPUs

- <https://www.mathworks.com/help/coder/ug/generate-digit-images-using-variational-autoencoder-intel-cpu.html>

■ AE / VAE various figures

- <https://www.mathworks.com/discovery/autoencoder.html>

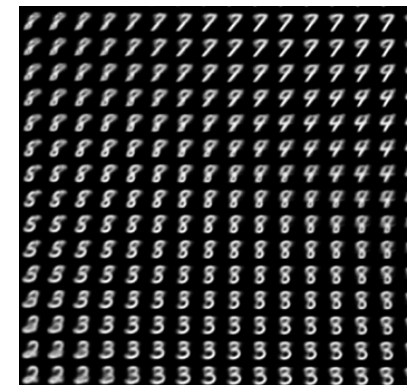


■ Anomaly detection using Variational Autoencoder(VAE)

- <https://www.mathworks.com/matlabcentral/fileexchange/73283-anomaly-detection-using-variational-autoencoder-vae>

■ Python VAE sites

- <https://keras.io/examples/generative/vae/>
- <https://www.assemblyai.com/blog/introduction-to-variational-autoencoders-using-keras/>
- <https://arxiv.org/pdf/1906.00446.pdf>
- <https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>
- <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>
- <https://www.analyticsvidhya.com/blog/2023/07/an-overview-of-variational-autoencoders/>



Outline

- Variational AE (VAE) vs. AE → The Roots of Generative AI

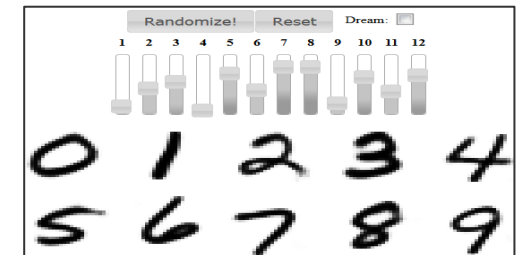
- VAE Architecture



- VAE Loss = KL Loss / Divergence + ELBO Loss

- Pseudo Code, Convolution / Dense, Sampling (how), Images / Numeric

- Walking in Latent Space
 - Better Controlled Generative AI

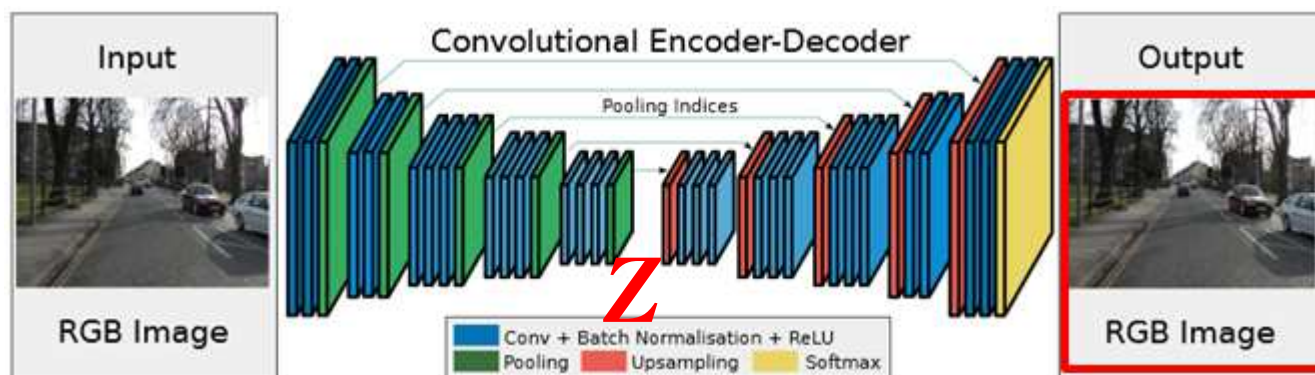


http://www.dpkimgma.com/sgyb_mnist_demo/demo.html



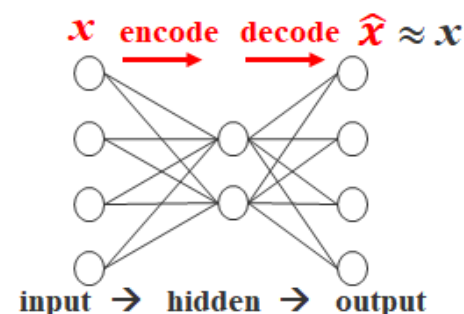
CNN Autoencoder vs. Variational Autoencoder

■ CNN Autoencoder

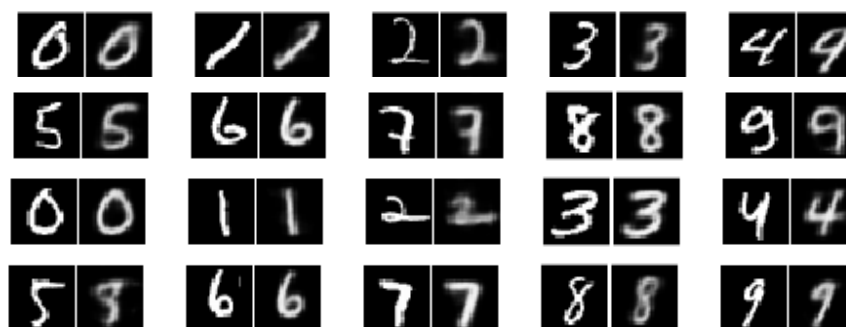


Autoencoder

can be deeper



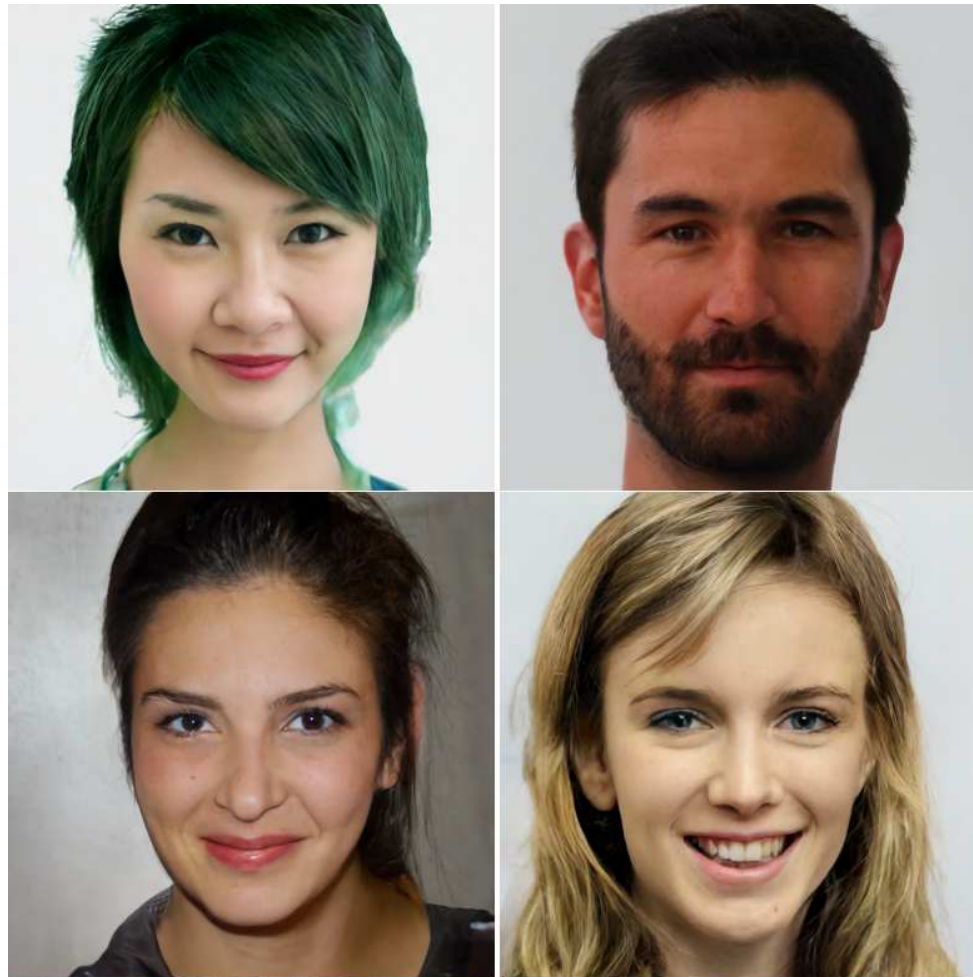
- One variant **Variational Autoencoder (VAE)** for generating images or signals (i.e. numbers)
 - A type of **generative AI** that is (supposed) easier to train / control than more recent GANs
 - Generation **w/ variations**



VAE Advanced Application

- Generate realistic looking faces that respect **long-range dependencies** such as matching *eye color* or symmetric facial features

<https://arxiv.org/pdf/1906.00446.pdf>
from Google



VAE Advanced Application / Zoom In

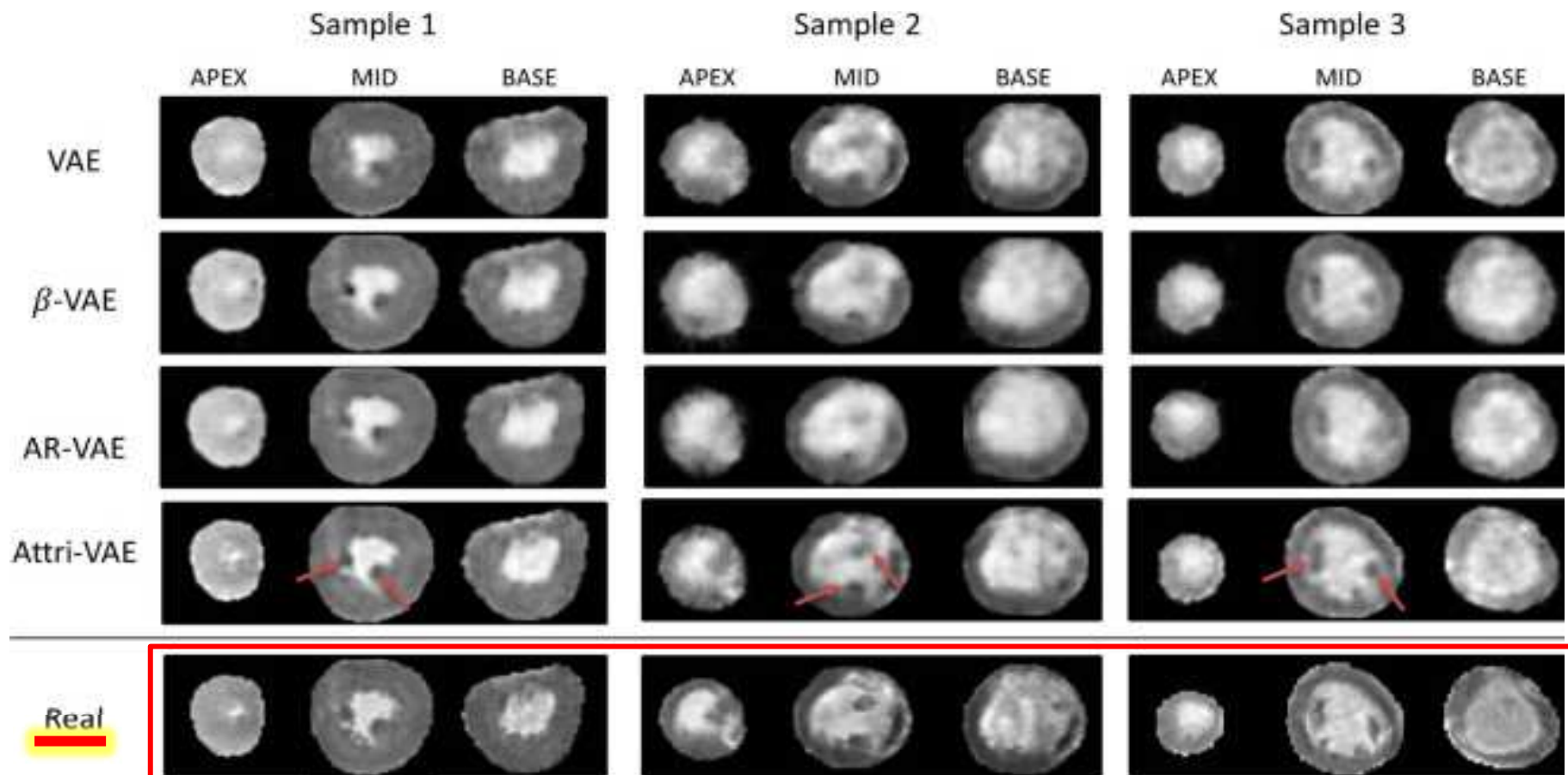
- Generate realistic looking faces that respect **long-range dependencies** such as matching *eye color* or symmetric facial features

<https://arxiv.org/pdf/1906.00446.pdf>
from Google 2019



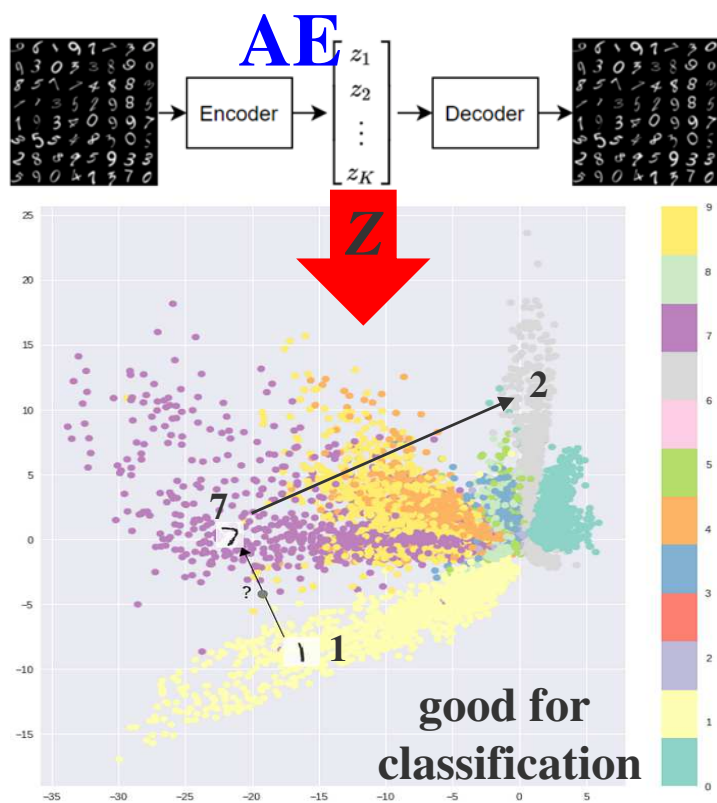
More Recent VAE in Medical Area

- Attri-VAE: Attribute-based interpretable representations of medical images with variational autoencoders, **2023**, <https://www.sciencedirect.com/science/article/pii/S0895611122001288>
 - Attribute-regularized: latent Z s are aligned w/ clinical/image attributes (e.g., age, cardiac vol.) for interpret
 - Attention maps: show how each attribute affects latent space and image features
 - Application: captures morphology and clinical info (i.e. Cardiac MRI)

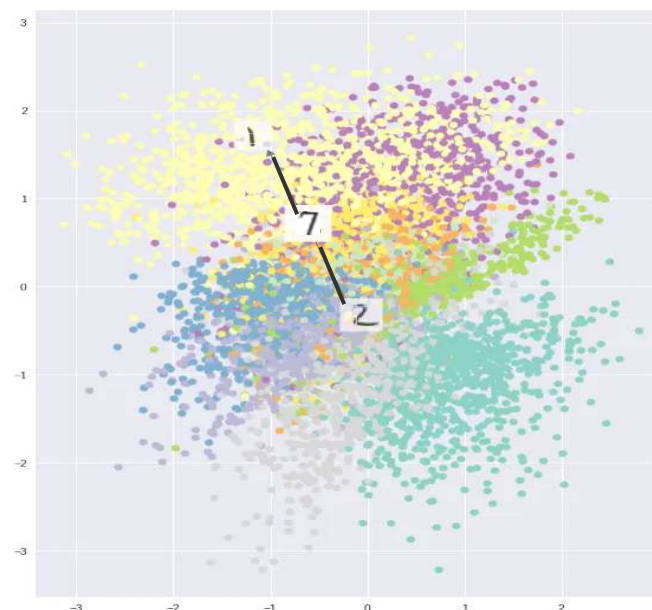


First AE Problem and What Else Do We Need?

- **Z** from AE below has good separation between classes → great for **classification**
- But bad from **generation** (i.e. generative AI) point of view
 - No similar structure for similar data (i.e. “1” & “7” or “2” & “7”) that shares similar looks
 - **Z** from AE may not allow easy interpolation / walking (i.e. digit 1 → 7, or 7 → 1.... $1 \Leftrightarrow 7 \Leftrightarrow 2$)
 - If similar **Zs** has more structured (more or less *continuous*), easier for sampling & interpolation



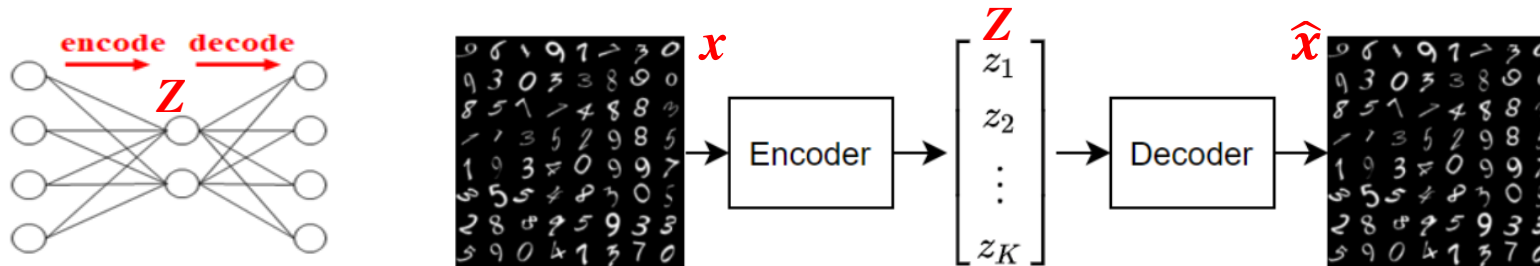
Z w/ better structures for...



<https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>

Second AE Problem & Variational Autoencoder (VAE)

- **AE** encodes input to low-D Z & decode (rebuild) Z from low-D to simulate input
- once trained
 - **Always** map same input X to **fixed** Z & **fixed** output $\hat{x}$, **NO** variation... like **overfitted**
 - **Always** encode same input to same output, **1:1 mapping**, **NOT** allow **ANY** variation

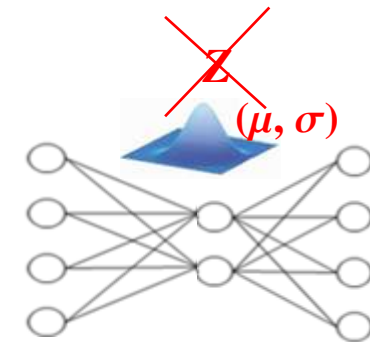


<https://www.mathworks.com/help/deeplearning/ug/train-a-variational-autoencoder-vae-to-generate-images.html>

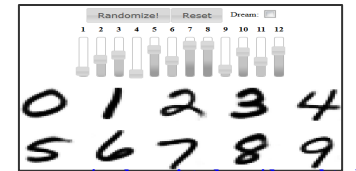
- **VAE** encodes a **dataset** into a **distribution** in latent space
 - i.e. VAE is **regularized** to avoid **overfitting** like in AE
 - $\approx$ mix deep learning with Bayesian inference (**probabilistic model**)



<https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>



Use 2 **Separated** & Interdependent: *Enc & Dec*



■ **Enc** NN enhances AE w/ **statistical** info



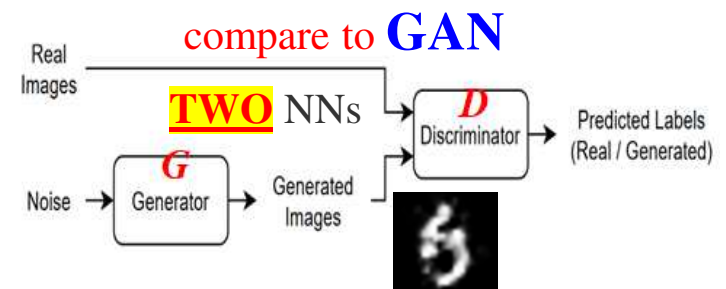
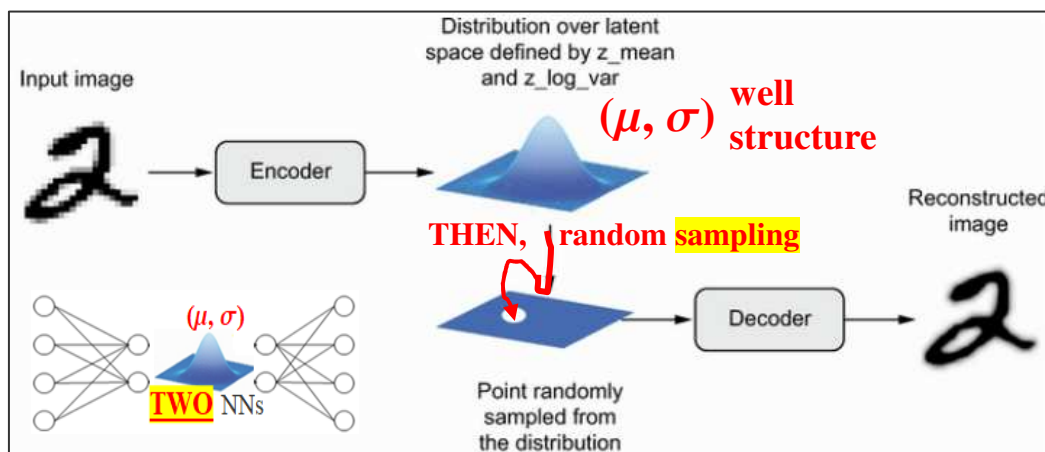
http://www.dpkimgma.com/sgvb_mnist_demo/demo.html

- Learn to turn (describe) entire dataset X into μ and σ : better structured latent Z spaces
- Imposing certain constraints / loss so VAE's **encoder** can learn Z as (μ, σ) from input X

■ **Dec** NN decodes ANY point randomly **sampled** in Z space to output (same in $\rightarrow$ **slightly different** out)

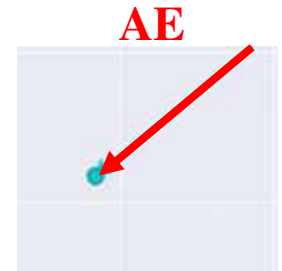
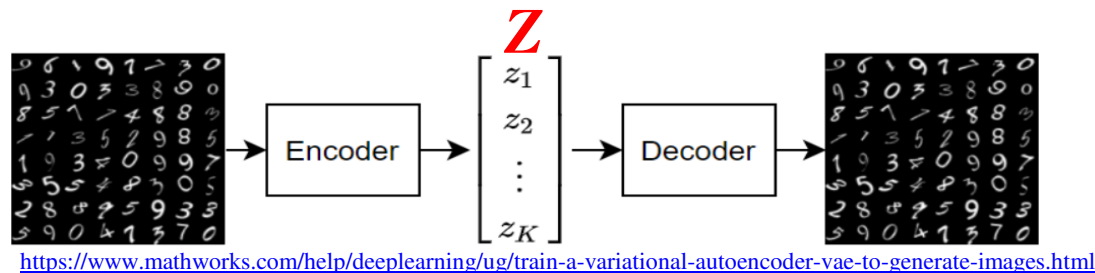
- Randomly sampled from μ and σ generated by **encoder**... then output by **rebuilding** input
- Each dim in Z vector has different μ and σ to capture / represent different latent meanings
- One app: get a Z vector of a *face* from $(\mu, \sigma)_{Enc}$ + “*smile vector*” $\rightarrow$ **Decoder** *smiley face*

■ Latent Z space under VAE's encoder is well structured, while **GAN** may not be

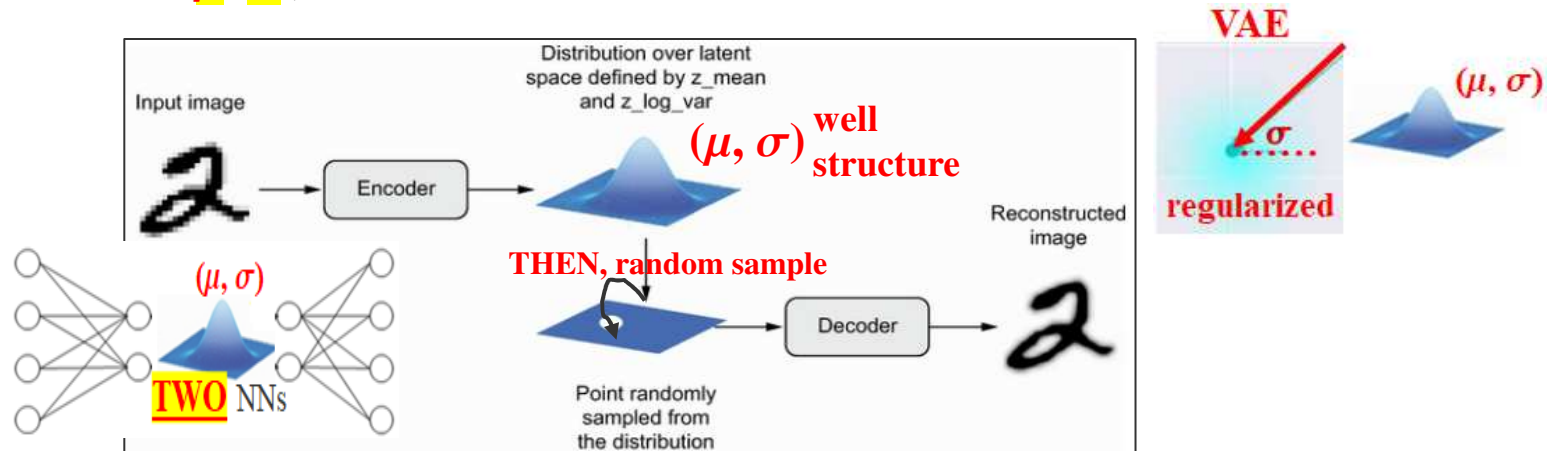


Summary, Train VAE to Generate Images (or **numeric** Data)

- **AE** encodes input to low-D $\mathbf{Z}$ & decode (rebuild) $\mathbf{Z}$ from low-D to simulate input
 - **Always** maps same input X to **fixed** $\mathbf{Z}$ & **fixed** output $\hat{x}$, **NO** variation... like **overfitted**
 - **Always** encode same input to same output, **1:1 mapping**, **NOT** allow **ANY** variation



- VAE **Encoder** NN learns probability **distribution** (μ, σ) to describe inputs
- VAE **Decoder** NN produces output by **randomly** sampled $\mathbf{Z}$ from **encoder's** learned distribution (μ, σ)



Outline

- Variational AE (VAE) vs. AE

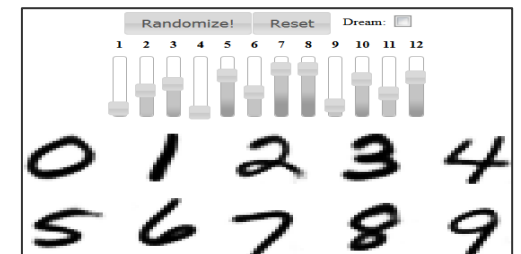
The Roots of Generative AI

- **VAE Architecture**

- VAE Loss = KL Loss / Divergence + ELBO Loss

- Pseudo Code, Convolution / Dense, Sampling (how), Images / Numeric

- Walking in Latent Space
 - Better Controlled Generative AI

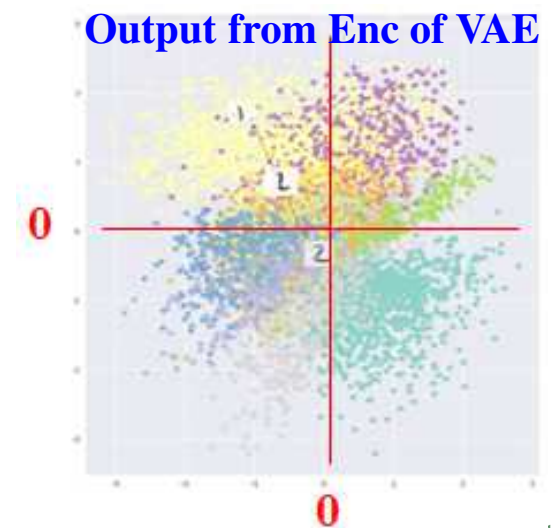
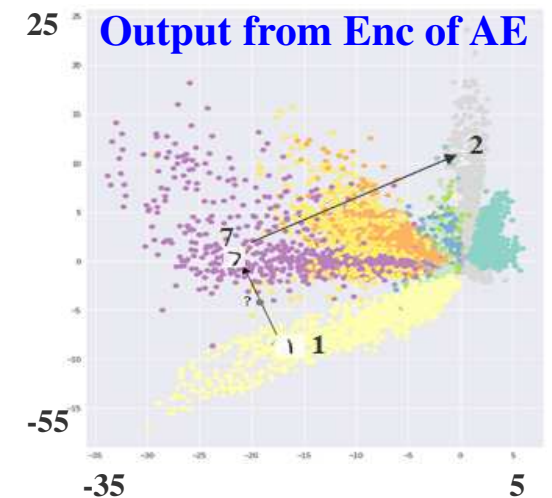
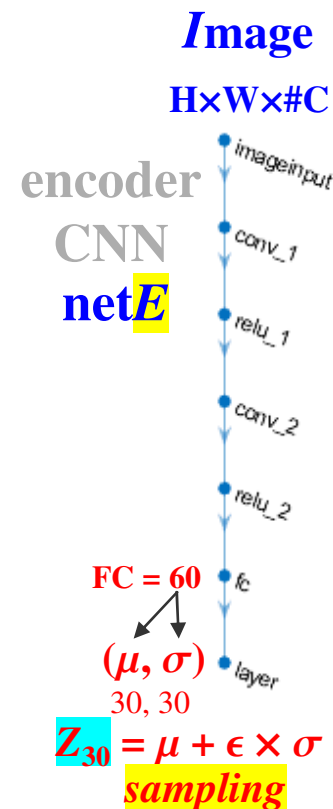
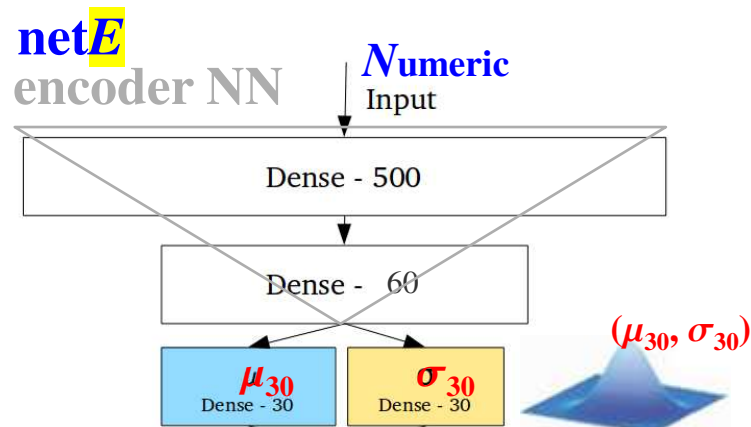


http://www.dpkgma.com/sgvb_mnist_demo/demo.html



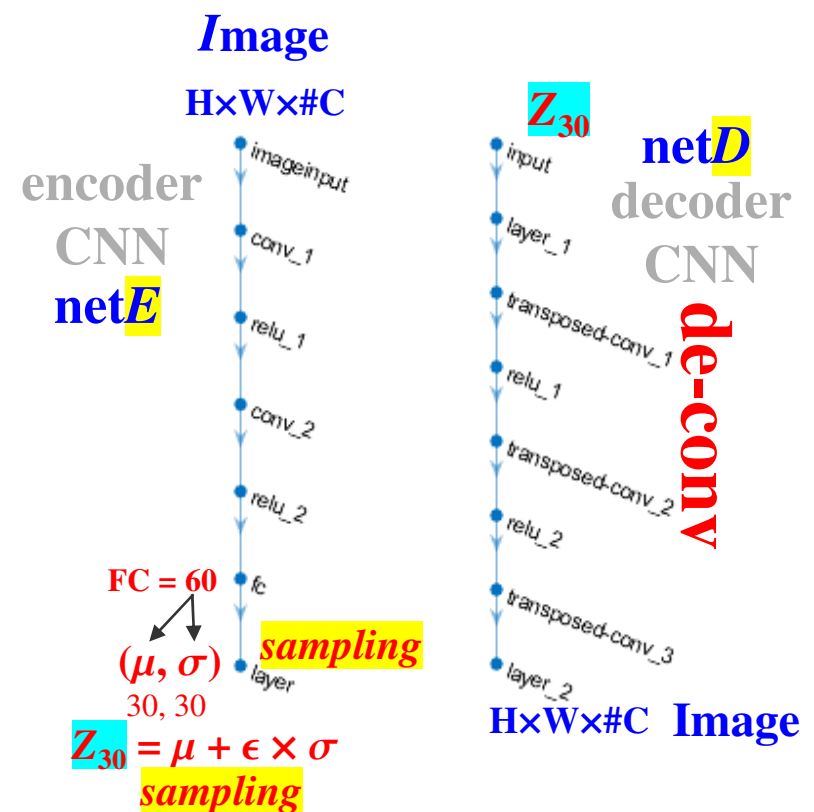
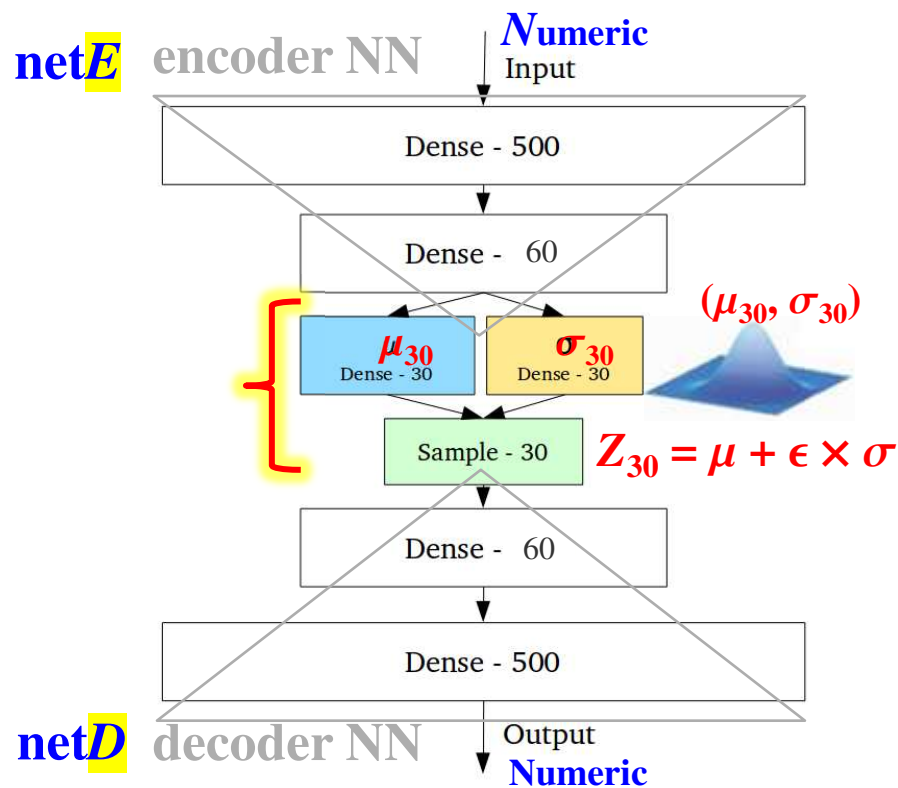
2 NNs in VAE Architecture, Layers in Encoder

- **Encoder** encodes input records, then outputs latent vectors (μ, σ)
 - Using a series of layers (either *dense* or *conv*) to compress data to a latent dimension (i.e. 30×2)
- **Decoder**..... randomly sample from...

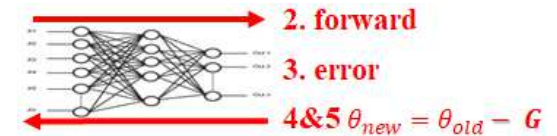


2 NNs in VAE Architecture, Encoder & Decoder

- **Encoder** encodes input records, then outputs latent vectors (μ, σ)
 - Using a series of layers (either *dense* or *conv*) to compress data to a latent dimension (i.e. 30×2)
- **Decoder samples** (??) and decodes latent vectors (μ, σ) to rebuild output $\approx$ input
 - Using a series of layers (either *dense* or *de-conv*) to de-compress Z back to input data



Pseudo Code for Training Encoder and Decoder



Loop epochs **# create your own loop**

shuffle(batches)

X = next(batch) **# get next batch**

lossVal, gradientE, gradientD = Loss(netE, netD, X)

update θ s **separately** in **netE, netD** $\theta_{new} = \theta_{old} - G$

End

function **Loss** **% collect info for ELBO loss**

mu, sigma, Z = predict(netE, X) $Z = \mu + \epsilon \times \sigma$

Rebuilt_X = predict(netD, Z)

THEN -----

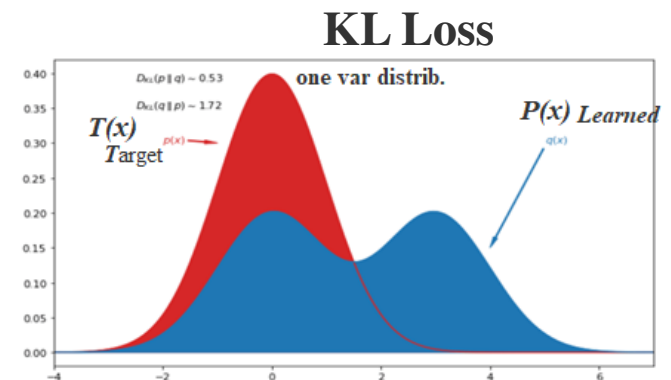
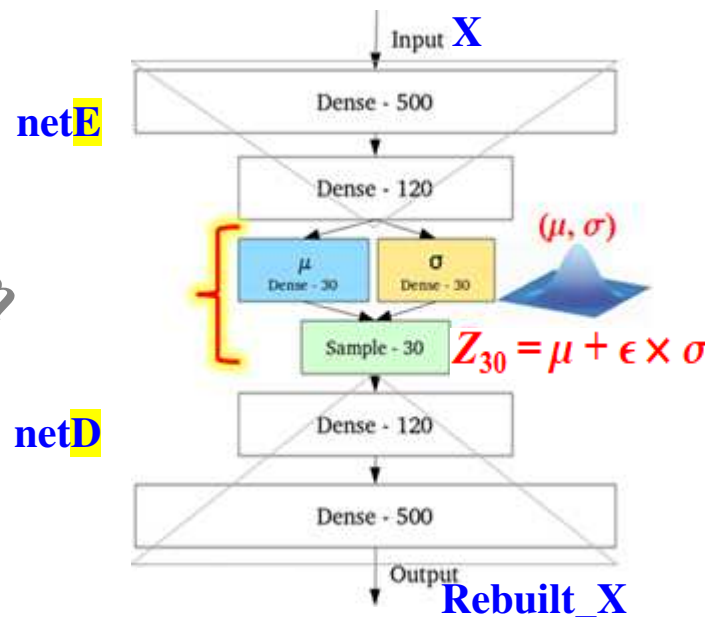
lossVal = elboLoss(Rebuilt_X, X, mu, sigma)

Backward gradient of **lossVal** against **netE/D.** θ s

```

while epoch < numEpochs && ~monitor.Stop
    epoch = epoch + 1;
    shuffle(mbg);
    while hasdata(mbg) && ~monitor.Stop
        iteration = iteration + 1;
        X = next(mbg);
        [loss, gradientsE, gradientsD] = dfeval(@modelLoss, netE, netD, X);
        [netE, trailingAvgE, trailingAvgSqE] = adamupdate(netE, ...
            gradientsE, trailingAvgE, trailingAvgSqE, iteration, learnRate);
        [netD, trailingAvgD, trailingAvgSqD] = adamupdate(netD, ...
            gradientsD, trailingAvgD, trailingAvgSqD, iteration, learnRate);
        recordMetrics(monitor, iteration, Loss=loss);
        updateInfo(monitor, Epoch=epoch + " of " + numEpochs);
        monitor.Progress = 100*iteration/numIterations;
    end
end
function [loss, gradientsE, gradientsD] = modelLoss(netE, netD, X)
    [Z, mu, logSigmaSq] = forward(netE, X);
    Y = forward(netD, Z);
    loss = elboLoss(Y, X, mu, logSigmaSq);
    [gradientsE, gradientsD] = dgradient(loss, netE.Learnables, netD.Learnables);
    function loss = elboLoss(Y, T, mu, logSigmaSq)
        reconstructionLoss = mse(Y, T);
        KL = -0.5 * sum(1 + logSigmaSq - mu.^2 - exp(logSigmaSq), 1);
        loss = reconstructionLoss + KL;
    end
    function Y = modelPredictions(netE, netD, mbg)
        Y = [];
        while hasdata(mbg)
            X = next(mbg);
            Z = predict(netE, X);
            XGenerated = predict(netD, Z);
            Y = cat(4, Y, extractdata(XGenerated));
        end
    end

```



■ VAE **ELBO** loss = Evidence Lower **BO**und..... breaking to **2 parts**... for **2 NNs**

● **ELBO** = **KL loss** + **rebuilt loss** = **KL loss** + **MSE**(generated output $\hat{X}$, input X)
Kullback-Leibler

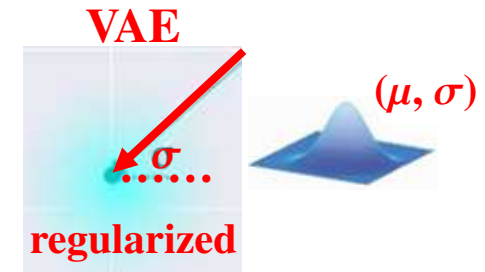
Outline

- Variational AE (VAE) vs. AE

The Roots of Generative AI

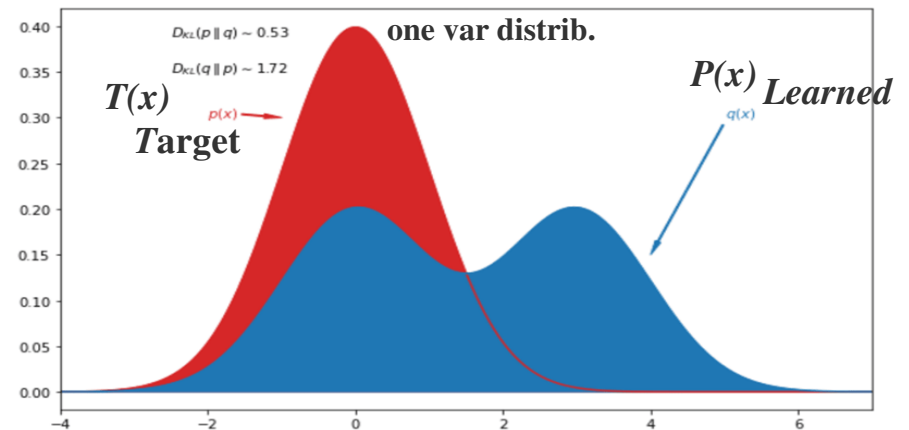
- VAE Architecture

- **VAE Loss = KL Loss / Divergence + ELBO Loss**



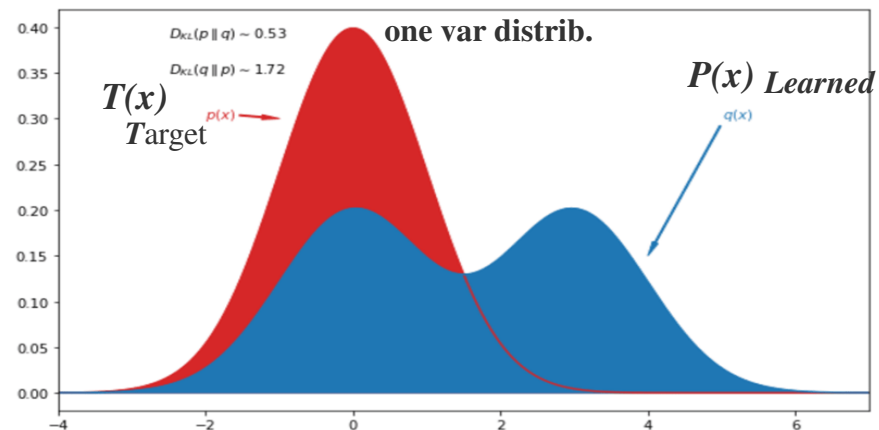
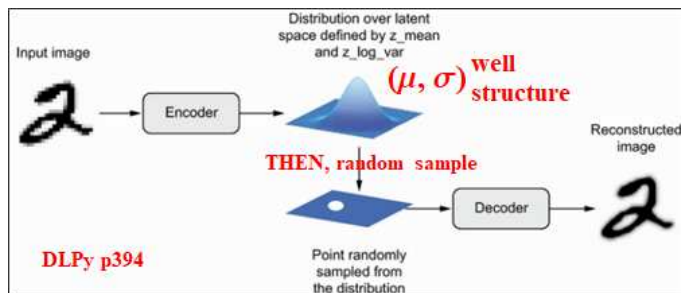
- Pseudo Code, Convolution / Dense, Sampling (how), Images / Numeric

- Walk in Latent Space
 - Better Controlled Generative AI



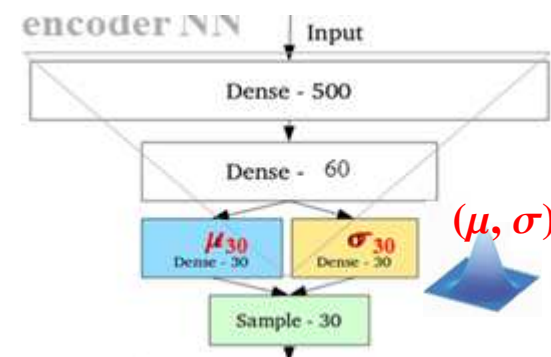
KL Divergence / Loss to Train Encoder to Generate Good (μ, σ)

- Need a **loss** function to guide encoder (also decoder) to learn good distribution (μ, σ) from input
 - idea = during training, compare *learned distribution* to a *target* (**normal**) *distribution* 
- **KL divergence** is commonly used to compare 2 distributions, $\sum \text{KL divergence}_{\text{BAT}} = \text{KL loss}$
 - = Measure how much 2 distributions *diverge* from each other 
 - VAE to **Min**(KL loss) $\approx$ optimize distrb. (μ, σ) on all dims (vars) to close to **normal** distrib.
 - Since normal distrb. = $(\mu_i = 0, \sigma_i^2 = 1)$, so KL loss is **min** when learning $\mu_i = 0, \sigma_i = 1$
 - **KL loss** = **KL divergence** = $\sum_{i=1}^D [\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1]$ for all **D**-var data... for normal distrb.
 - $-\sum P(x) \log\left(\frac{P(x)}{T(x)}\right)$ P is predicted w.r.t. T target... for any distrb. w/o assuming Gaussianity



Visual of Various Degrees of **KL Divergence** (between 0 to ∞)

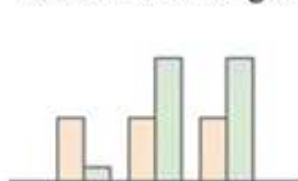
- **KL divergence** is commonly used to compare 2 distributions, $\Sigma \text{KL divergence}_{\text{BAT}} = \text{KL loss}$
 - = Measure how much 2 distributions **diverge** from each other
 - VAE to **Min(KL loss)** = optimize distrib. **(μ, σ)** on all dims (vars) to close to **normal** distrib.
 - KL loss is minimal when $\mu_i = 0, \sigma_i = 1$ i.e. $N(\mu_i = 0, \sigma_i^2 = 1)$
 - = learn (μ, σ) to be close to normal distribution
 - **KL loss = KL divergence** = $\sum_{i=1}^D [\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1]$ for D -var



High KL divergence



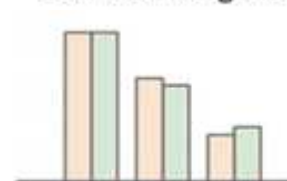
Medium KL divergence



Low KL divergence



Low KL divergence

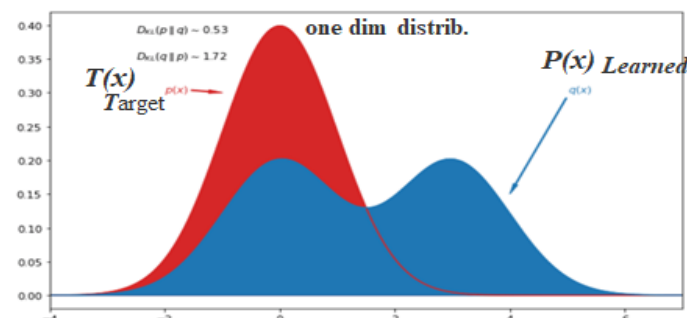
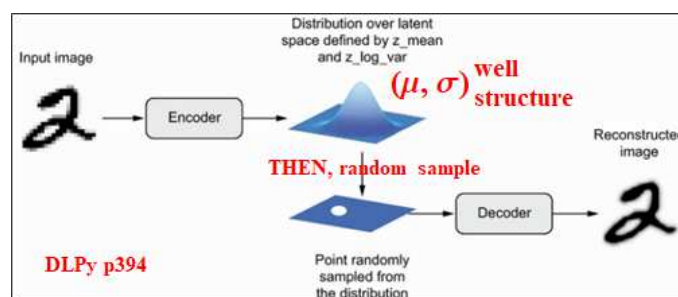
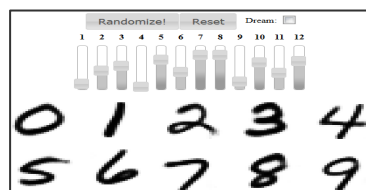


any **Target distrib.**

learned P distrib.

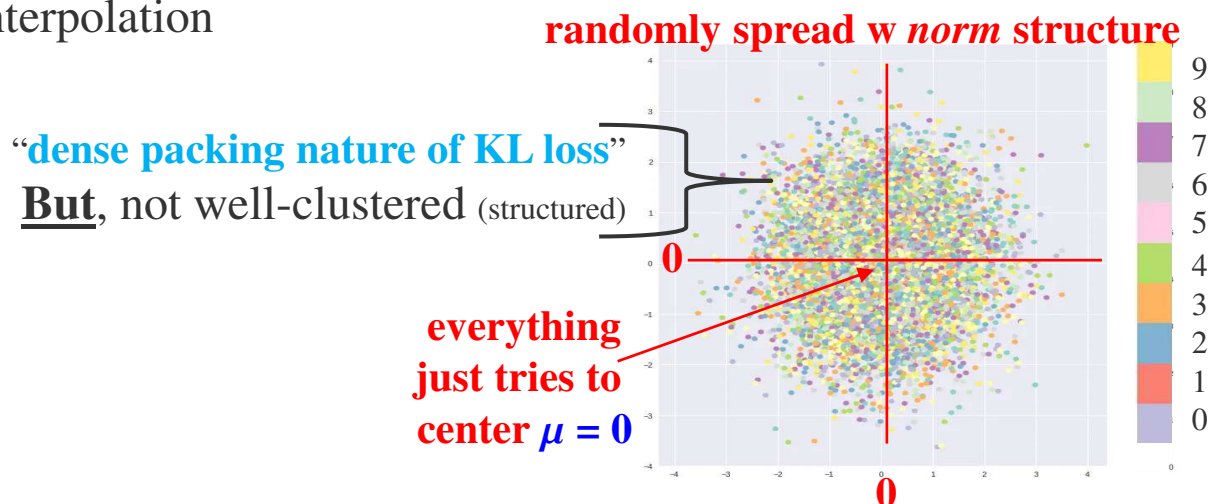
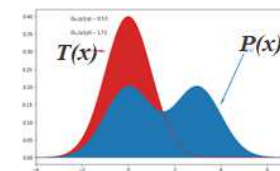
**i.e. market shifting,
sensor deviation...**

<https://medium.com/octavian-ai/a-simple-explanation-of-the-inception-score-372dff6a8c7a#:~:text=What%20is%20the%20Inception%20score,%5Bof%20image%20quality%5D%E2%80%9D.>



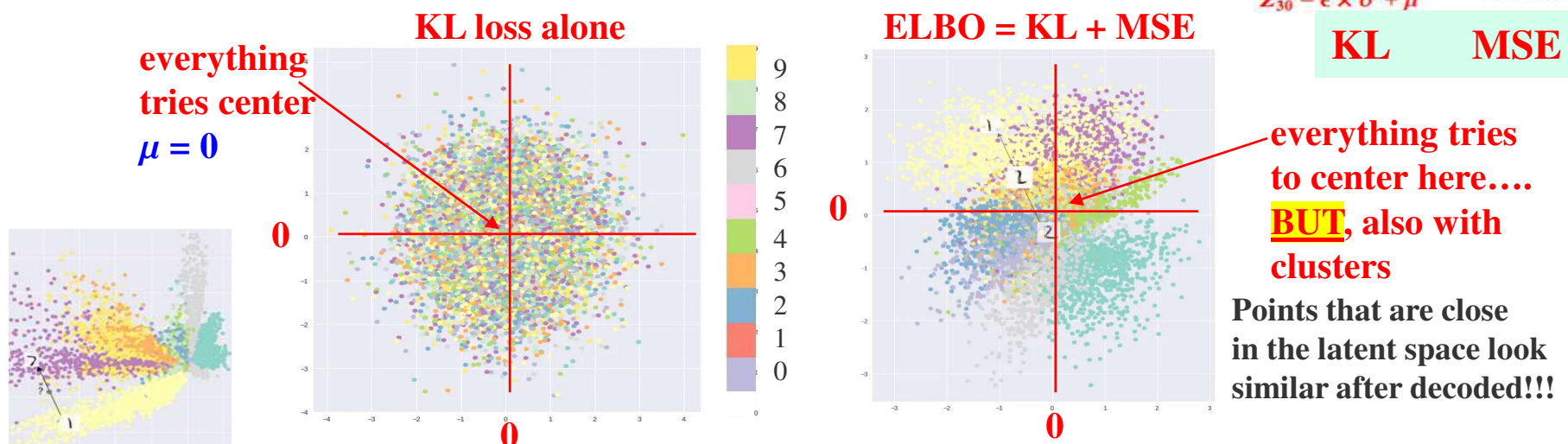
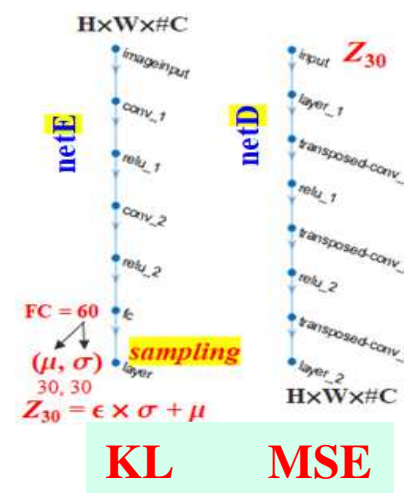
BUT... KL Loss Alone For Encoder Is **NOT** Enough

- **KL loss** guides **encoder** to learn good distribution (μ, σ) from input
 - idea = during training, compare learned distribution to a target (normal) distribution
 - **KL loss** = $\sum_{i=1}^D \sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1$ for D -dimension data
 - **KL loss** is minimal when $\mu_i = 0, \sigma_i = 1$ i.e. target = normal distribution $N(\mu_i = 0, \sigma_i^2 = 1)$
- **BUT**, KL loss **alone** may lead the learned **Zs** **randomly** spread around μ in **Z-space**
 - Norm-structured, BUT **Z** is still not easy for interpolation (i.e. $1 \rightarrow 7, 7 \rightarrow 1$ OR $1 \Leftrightarrow 7 \Leftrightarrow 2$)
- What we need is.....
 - Similar inputs $\rightarrow$ similar **Zs** w/ **similar structured** (more or less continuous) around (μ, σ)
 - Easier for sampling & interpolation
 - What do I mean??
 - HOW???



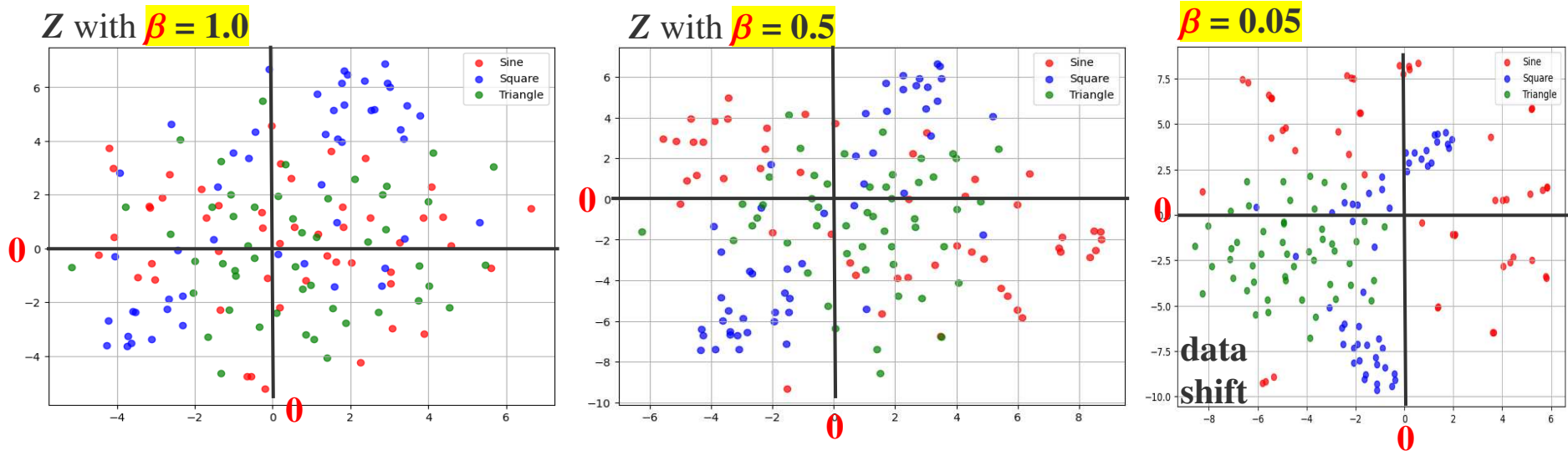
From **KL Loss Alone** to Combined **ELBO** Loss

- Add **rebuilt error** to coordinate **Enc & Dec** to generate output that is similar to input
- VAE final loss = **ELBO** *E*vidence *L*ower *B*ound
 - **ELBO** = **KL loss** + **rebuilt loss** = **KL loss** + **MSE**(generated output $\hat{X}$, input X)
 - **ELBO** = “dense packing nature of KL loss” + “cluster nature of rebuilt loss”
- **KL loss** is to constrain / teach **encoder** & decoder
- **Rebuilt loss** is to constrain / teach **decoder** & encoder
- **Encoder** produces μ and $\log(\sigma^2)$ and give them to **decoder** to compute / assemble the final **ELBO** loss

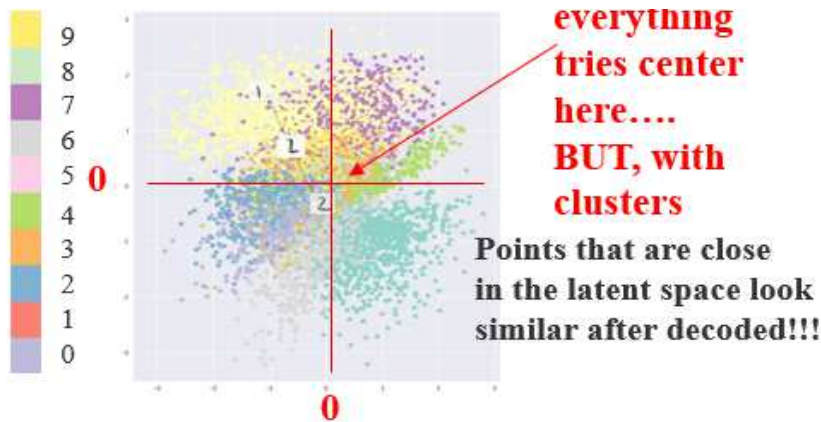
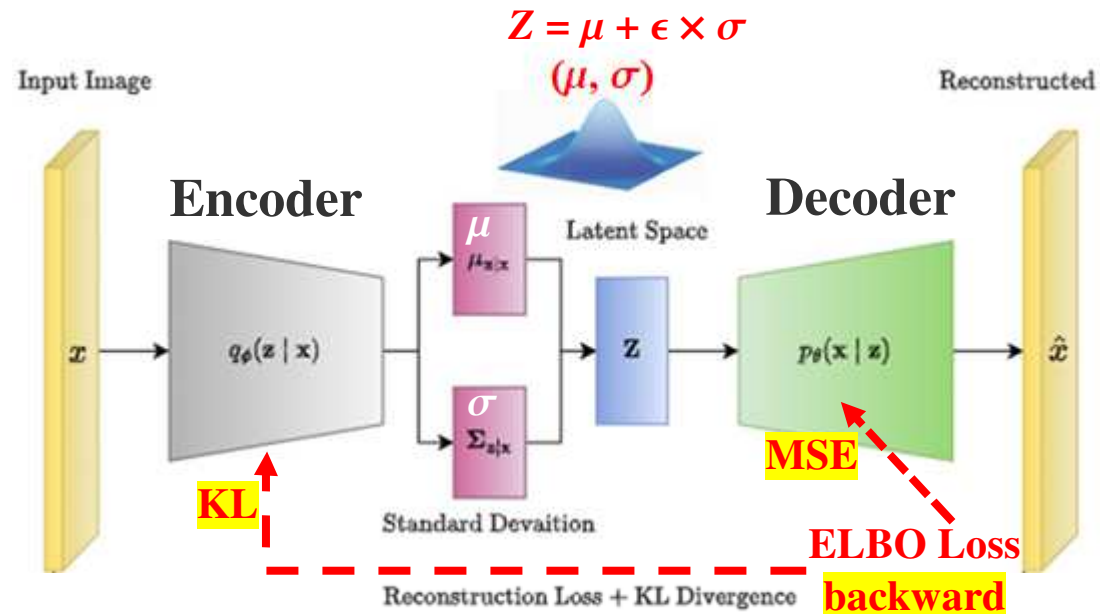


Customizing Weight for Losses

- Add **rebuild error** to coordinate **Enc & Dec** to generate output that is similar to input
- VAE final loss = **ELBO** *E*vidence *L*ower *B*ound
 - **ELBO** = **KL loss** + **rebuild loss** = **KL loss** + **MSE**(generated output $\hat{X}$, input X)
 - **ELBO** = “dense packing nature of KL loss” + “cluster nature of rebuild loss”
- VAE **ELBO** = $\beta \times$ **KL loss** + **rebuild loss**

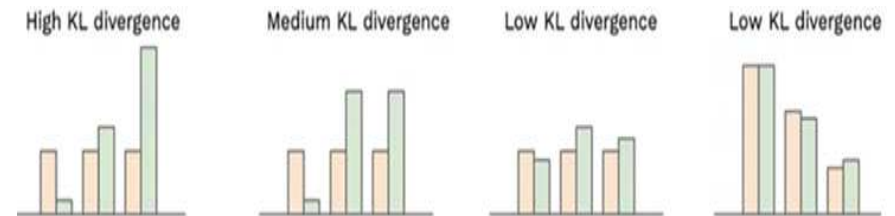


Visual of VAE with TWO Losses



$$\text{ELBO} = \beta \times \text{KL loss} + \text{rebuild loss}$$

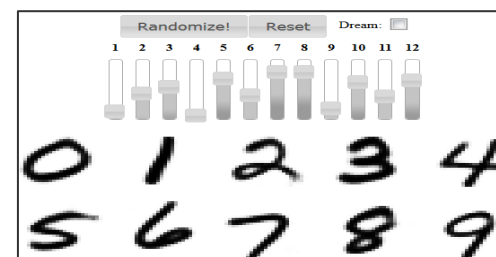
β = balance factor



- Training NN with KL loss, sometime, may have potential vanish gradient (**Appendix**)

Outline

- Variational AE (VAE) vs. AE, **The Roots of Generative AI**
- VAE Architecture
- VAE **Loss** = KL Loss / Divergence + ELBO Loss
- **Pseudo Code, Convolution / Dense, Sampling (how), Images / Numeric**
- Walking in Latent Space
 - Better Controlled Generative AI



http://www.dpkgingma.com/sgvb_mnist_demo/demo.html



VAE Implementation Pseudo Code

```

Loop epochs    # create your own loop
  shuffle(batches)
  X = next(batch)
  lossVal, gradientE, gradientD = Loss(netE, netD, X)
  update  $\theta$ s separately in netE, netD   $\theta_{\text{new}} = \theta_{\text{old}} - G$ 
End
    
```

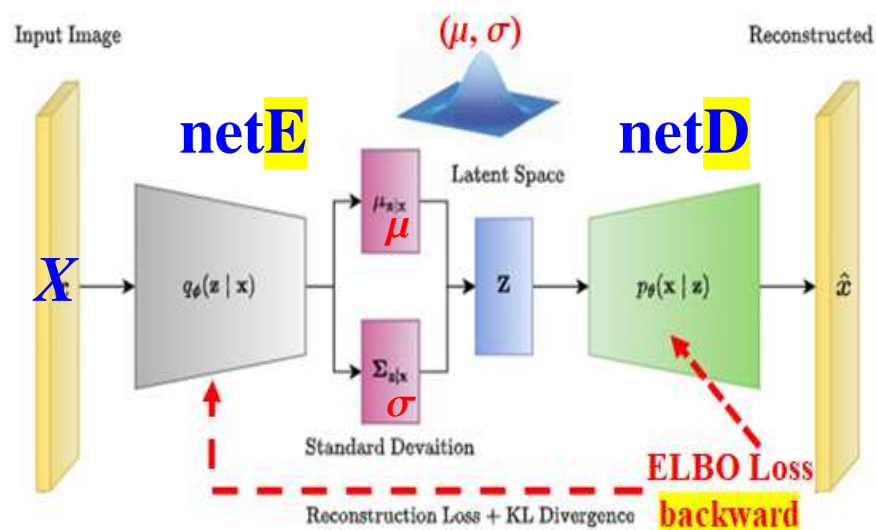
```

function Loss
  mu, sigma, Z = predict(netE, X)  # sampling
  Rebuilt_X = predict(netD, Z)
  lossVal = elboLoss(Rebuilt_X, X, mu, logSigma)  # log-variance
  Backward gradient of lossVal against netE/D. $\theta$ s
    
```

```

function elboLoss(Rebuilt_X, X, mu, logSigma)
  rebuiltLoss = mse(Rebuilt_X, X)  # rebuilt loss

  # KL divergence
  KL = -0.5 * sum(1 + logSigma - mu.^2 - exp(logSigma), 1);
  KL = mean(KL);
   $\beta??$ 
  loss = rebuiltLoss + KL;
  KL loss =  $\sum_{i=1}^n \sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1$ 
  ELBO = KL loss + rebuilt loss
    
```



**BUT, after training,
how to generate a new output??**

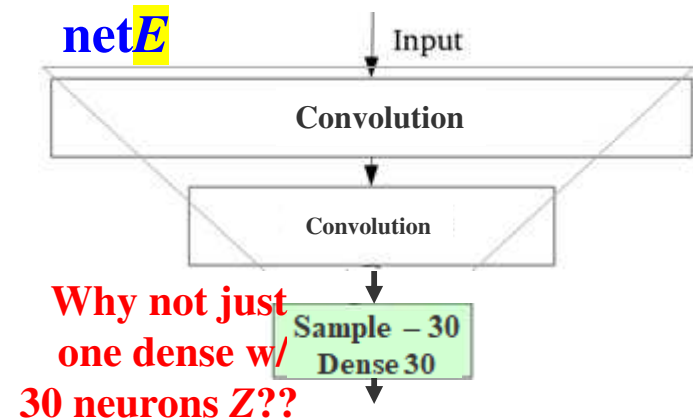
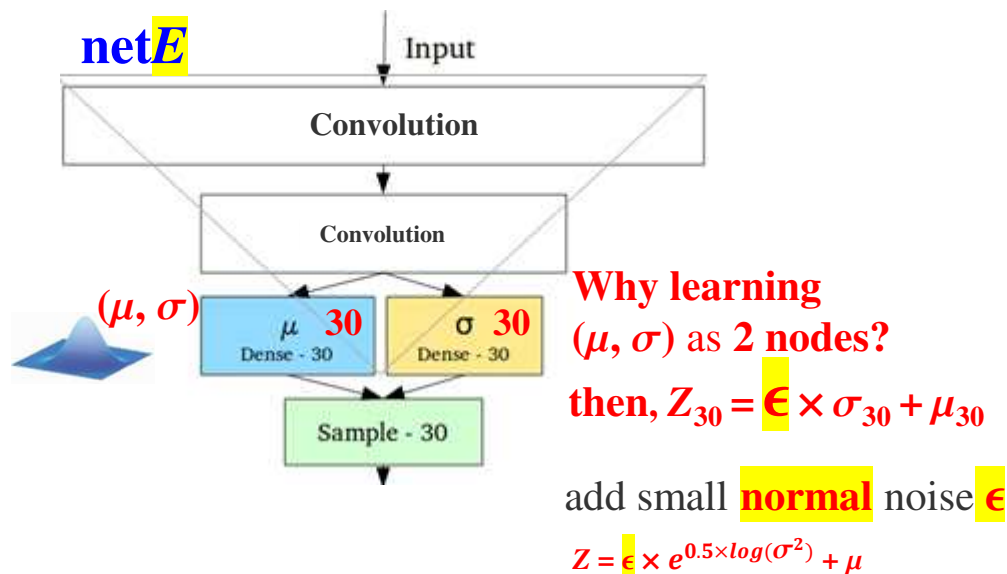
```

1. select input X  2. random sampling error  $\epsilon$ 
mu, sigma, Vec_Z = predict(netE, X,  $\epsilon$ ) for encoder
Gen_Out = predict(netD, Vec_Z) for decoder
    
```

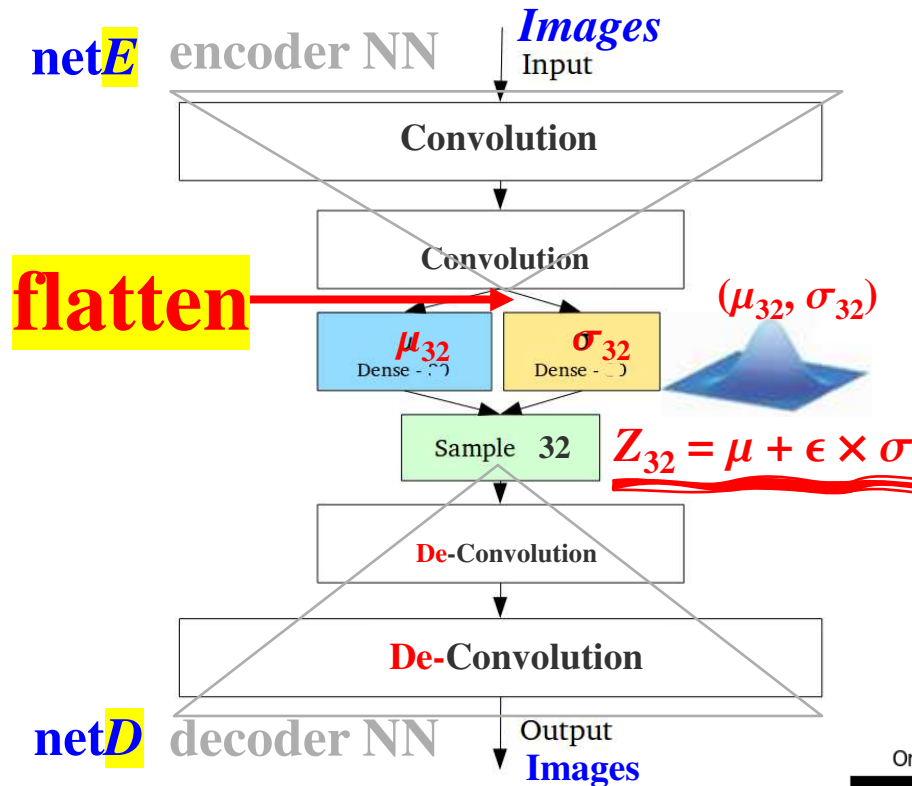
Why Adding Normal Random Noise In Encoder **Sampling**??

$$Z = \epsilon \times \sigma + \mu$$

- Prevent VAE memorize (**overfit**) training data too much (i.e. regularization)
 - Noise encourages VAE to search/gen over different regions in latent space
 - Prevent VAE stuck in a small area in latent space
 - Prevent **overfit** where outputs $\approx$ training input, or
 - Prevent **mode collapse**: outputs $\neq$ training but most outputs are similar (i.e. fix in a **useless** area)
- Normal noise makes Z more normal distrib. lead to more robust & better generation
- Refer to as **reparameterization** $\rightarrow$ else non-differentiable from concept of $Z \sim N(\mu, \sigma^2)$



My VAE Using 30 MNIST Digits (0, 1, 2)



0
1
2

Data_DigitImg_012_Each10_InSubFolders.zip

σ = standard deviation, σ^2 = variance

VAR actually learns $= \log(\sigma^2)$, log variance

σ^2 either very small or very large

log-space compresses that range

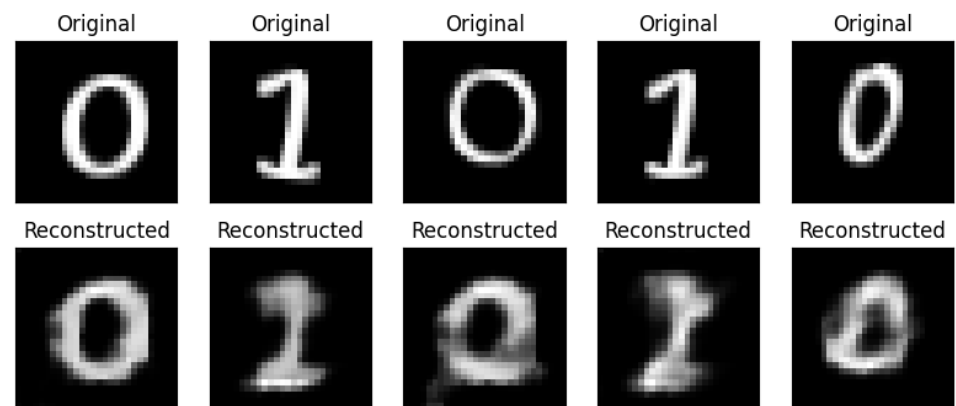
$$\sigma = \sqrt{\sigma^2} = \sqrt{e^{\log(\sigma^2)}}$$

$$= (e^{\log(\sigma^2)})^{1/2} = e^{0.5 \times \log(\sigma^2)}$$

VAI_Slide_VAE_CNN_Digits.ipynb

■ Using All (1,000) MNIST Digits

- <https://keras.io/examples/generative/vae/>



Implementation at High-Level

VAI_Slide_VAE_CNN_Digits.ipynb

```
def Encoder(x): # 1.
    h = NN_E_layers(x)
    mu = Dense(h, Z_dim)
    log_var = Dense(h, Z_dim)
    return mu, log_var
```

```
def SampleZ(mu, log_var): # 2.
    sigma = exp(0.5 * log_var)
    epsilon = rand_norm(shape= Z_dim)
    Z = mu + sigma * epsilon
    return Z
```

```
def Decoder(Z): # 3.
    x_rebuilt = NN_D_layers(Z)
    return x_rebuilt
```

4. build E & D models, individually

```
vae_in = Input(...)
mu, log_var = Encoder(vae_in)
E = Model(vae_in, [mu, log_var])
```

```
z_in = Input(Z_dim)
recon = Decoder(z_in)
D = Model(z_in, recon)
```

5. connect together

```
mu, log_var = E(vae_in)
Z = Lambda(SampleZ)([mu, log_var]) # repair
Xrecon = D(Z)  $Z = \mu + \epsilon \times \sigma$ 
```

6. build full VAE, set output order

```
vae = Model(vae_in, [mu, log_var, Xrecon])
z_extractor = Model(vae_in, Z) # for walking
```

```
for each batch x_bat: # 6. training
    with GradientTape():
        # use pre-def output order
        mu, log_var, Xrecon = vae(x_bat)
        loss = VAELoss(x_bat,
                        mu, log_var, Xrecon)

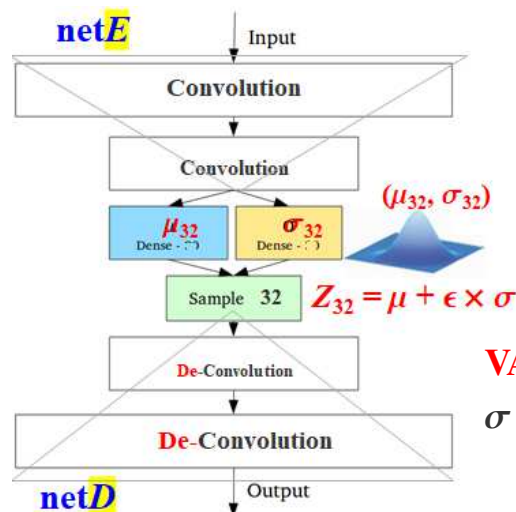
        gradients = compute_gradients(loss, vae.θs)
        optimizer . apply(gradients)
```

```
function VAELoss(x_bat, mu, log_var, Xrecon):
    recon_loss = MSE(x_bat, Xrecon)

    kl_loss = -0.5 * sum(1 + log_var - mu^2 - exp(log_var))

    total_loss = recon_loss + beta * kl_loss
```

```
input x
↓
Encoder
↓
(mu, log_var)
↓
Sample Z = mu + sigma * epsilon
↓
Decoder
↓
reconstruction Xrebuilt
↓
loss = MSE + KL
↓
Backprop & update all θs in D/E
```



X $Z \sim N(\mu, \sigma^2)$

VAR learns $\log(\sigma^2)$, log variance

$\sigma = e^{0.5 \times \log(\sigma^2)}$

VAE, Customize Training Loop w/ Custom Loss Function

VAI_Slide_Own_TrainLoop_XEntropy.ipynb

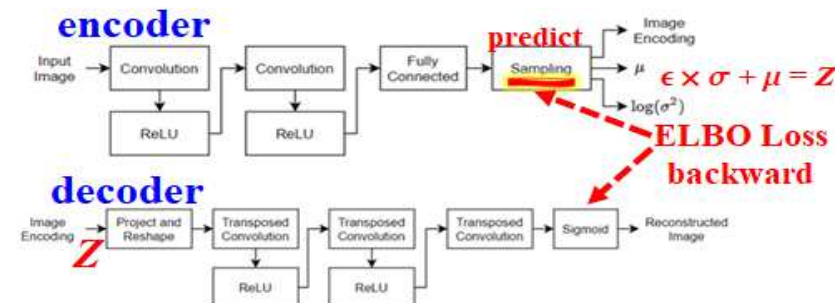
General Flow

```
def train_BAT(model, optimizer, YourLoss_fn, xBat, yBat):
    # Record steps & orders in compute (x, yhat, loss) for gradient
    with tf.GradientTape() as tape:
        # Forward pass: model predicts on one batch
        yhat = model(x_batch, training=True)

    # Compute loss you need check BAT ≠ NONE
    loss_value = YourLoss_fn(y_batch, yhat)

    # Compute gradient of loss w.r.t θs,  $G = \partial(\text{loss}) / \partial(\theta_s)$ 
    Gs = tape.gradient(loss_value, model.trainable_weights)
    # Use optimizer to update θs,  $\theta_{\text{new}} = \theta_{\text{old}} - G$ 
    optimizer.apply_gradients(zip(Gs, model.trainable_weights))

    return loss_value
```



VAE

VAI_Slide_VAE_CNN_Digits.ipynb

VAI_Slide_VAE_Digits_Manipulate.m

```
def train_step_fn(vae, optimizer, xBat):
    # Record steps & orders in compute gradient
    with tf.GradientTape() as tape:
        # Forward VAE returns: [reconstruct, z_mean, z_log_var]
        rebuildX, z_mean, z_log_var = vae(xBat)

    # Compute 2 losses
    rebuild_loss = Rebuild_loss_fn(xBat, rebuildX)
    kl_loss = kl_loss_fn(z_mean, z_log_var)

    total_loss = rebuild_loss + kl_loss

    # Compute gradient of loss w.r.t θs,  $G = \partial(\text{loss}) / \partial(\theta_s)$ 
    Gs = tape.gradient(total_loss, vae.trainable_weights)
    # Use optimizer to update θs,  $\theta_{\text{new}} = \theta_{\text{old}} - G$ 
    optimizer.apply_gradients(zip(Gs, vae.trainable_weights))

    return total_loss, rebuild_loss, kl_loss
```

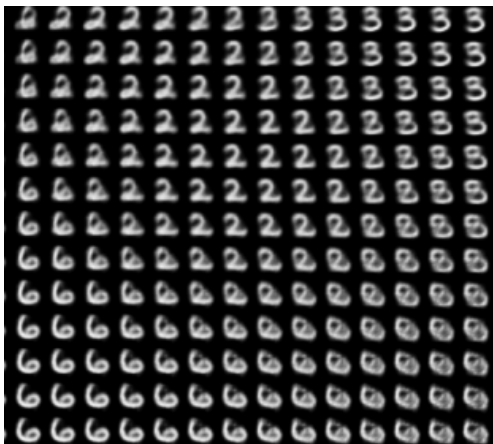
$$\text{KL loss} = \sum_{i=1}^n \sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1$$

$$0.5 * \text{sum}(\text{sigma}^{**2} + \text{mu}^{**2} - \log(\text{sigma}^{**2}) - 1)$$

VAE Using All (1,000) MNIST Digits <https://keras.io/examples/generative/vae/>

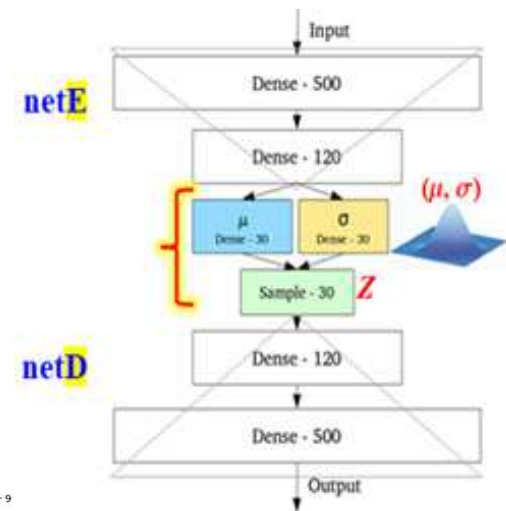
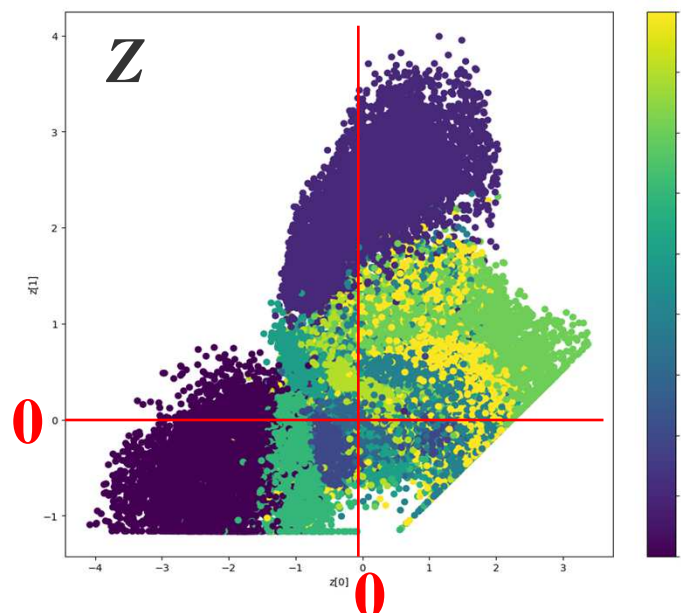
Encoder

Layer (type)	Output Shape
input_layer (InputLayer)	(None, 28, 28, 1)
conv2d (Conv2D)	(None, 14, 14, 32)
conv2d_1 (Conv2D)	(None, 7, 7, 64)
flatten (Flatten)	(None, 3136)
dense (Dense)	(None, 16)
z_mean (Dense)	(None, 2) μ
z_log_var (Dense)	(None, 2) $\log(\sigma^2)$
sampling (Sampling)	(None, 2)



Decoder

Layer (type)	Output Shape
input_layer_1 (InputLayer)	(None, 2)
dense_1 (Dense)	(None, 3136)
reshape (Reshape)	(None, 7, 7, 64)
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)



$$\text{VAE ELBO} = \text{KL loss} + \text{rebuild loss}$$

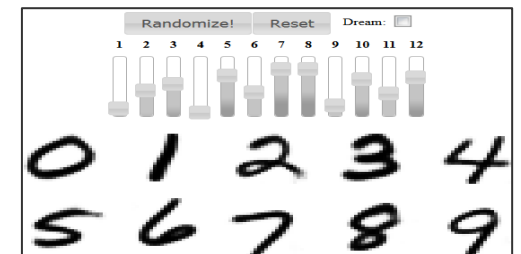
Outline

- Variational AE (VAE) vs. AE
- VAE Architecture
- VAE Loss = KL Loss / Divergence + ELBO Loss

The Roots of Generative AI

- **Pseudo Code, Convolution / Dense, Sampling (how), Images / Numeric**

- Walking in Latent Space
 - Better Controlled Generative AI



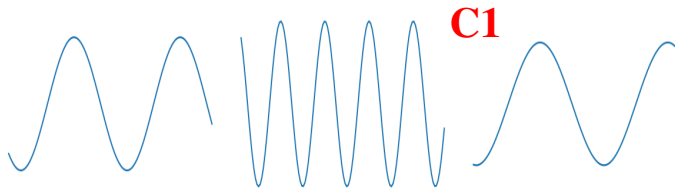
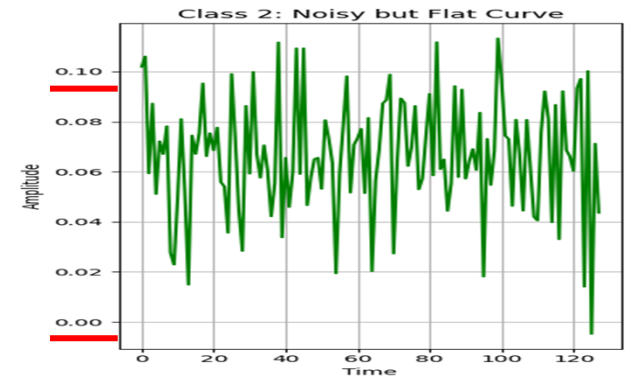
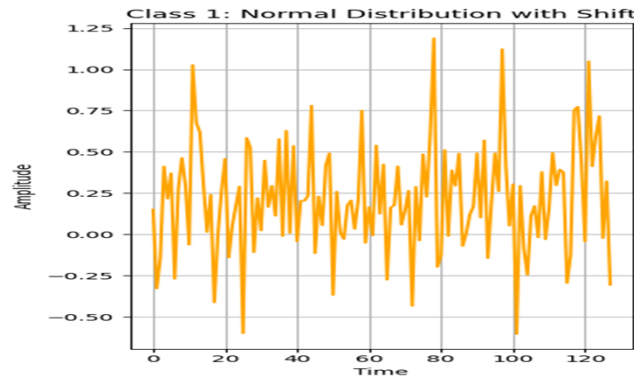
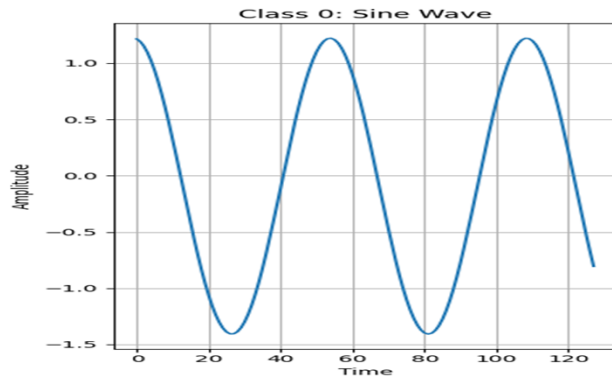
http://www.dpkgma.com/sgvb_mnist_demo/demo.html



VAE for Numeric Dataset

VAI_Slide_VAE_Signals_3Types_Beta_1_0.ipynb

- Manually generate 3 classes of curves (signals), 50 each... remember **Z-Score** the data!!!
 - i.e. stock market, vital sign, #disease cases overtime, sensors, etc.,



Encoder

InputLayer	(None, SIGNAL_LENGTH)	128
Dense (100, relu)	(None, 100)	
Dense (85, relu)	(None, 85)	
Dense (z_mean)	(None, LATENT_DIM)	64
Dense (z_log_var)	(None, LATENT_DIM)	64
Sampling	(None, LATENT_DIM)	64

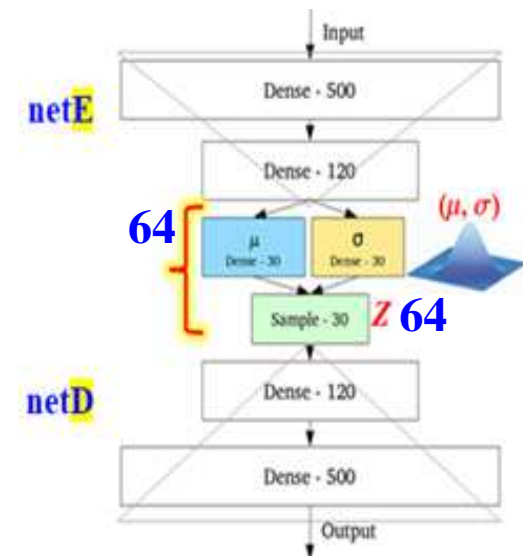
Decoder

InputLayer	(None, LATENT_DIM)	64
Dense (85, relu)	(None, 85)	
Dense (100, relu)	(None, 100)	
Dense (output)	(None, SIGNAL_LENGTH)	128

SIGNAL_LENGTH = 128

LATENT_DIM = 64

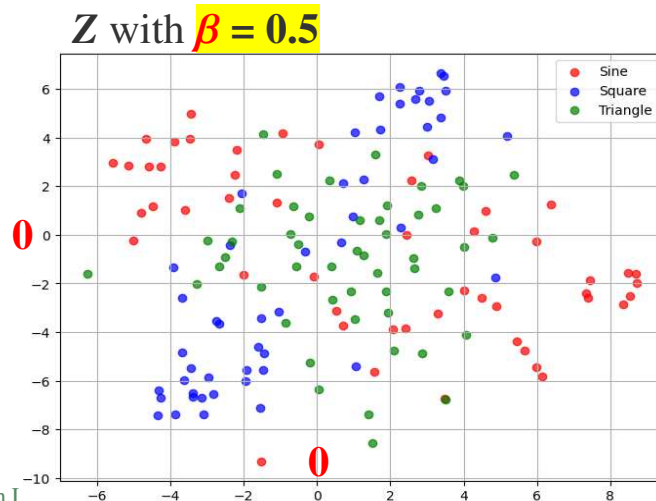
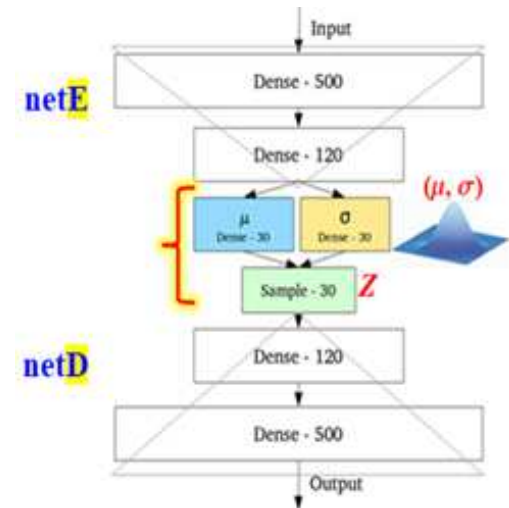
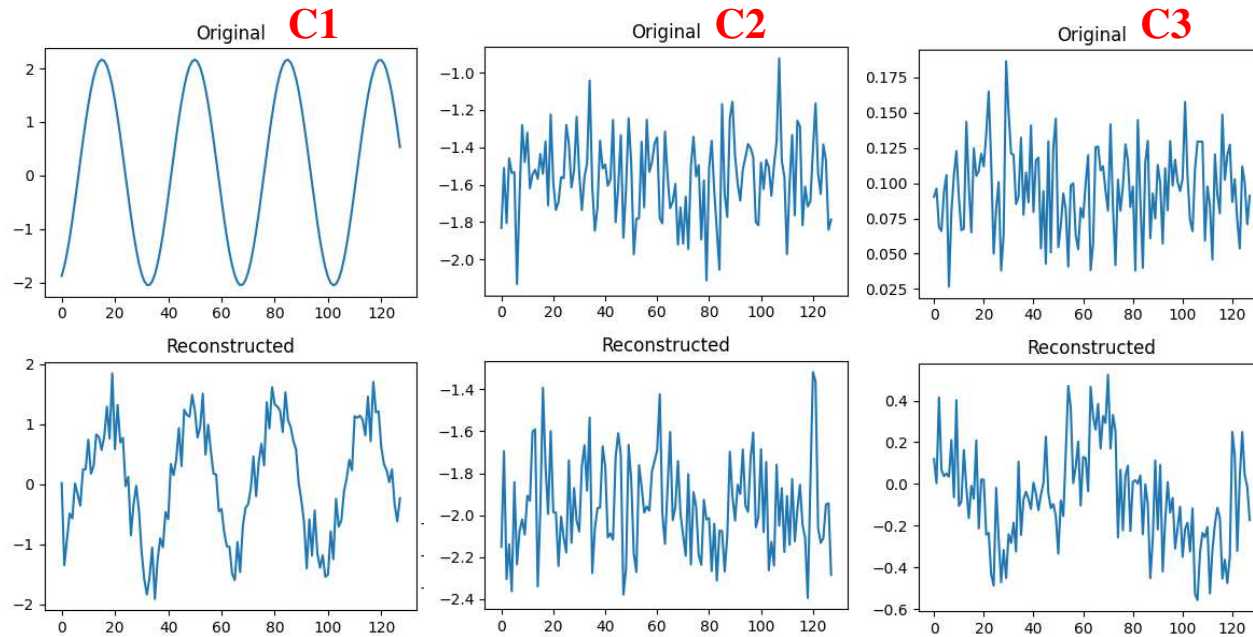
Epoch = 300 (short training time)



VAE for Numeric Dataset, With $\beta = 0.5$

■ VAE **ELBO** = $\beta \times \text{KL loss}$ + rebuilt loss

$$\text{KL} = \sum_{i=1}^D \sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1$$



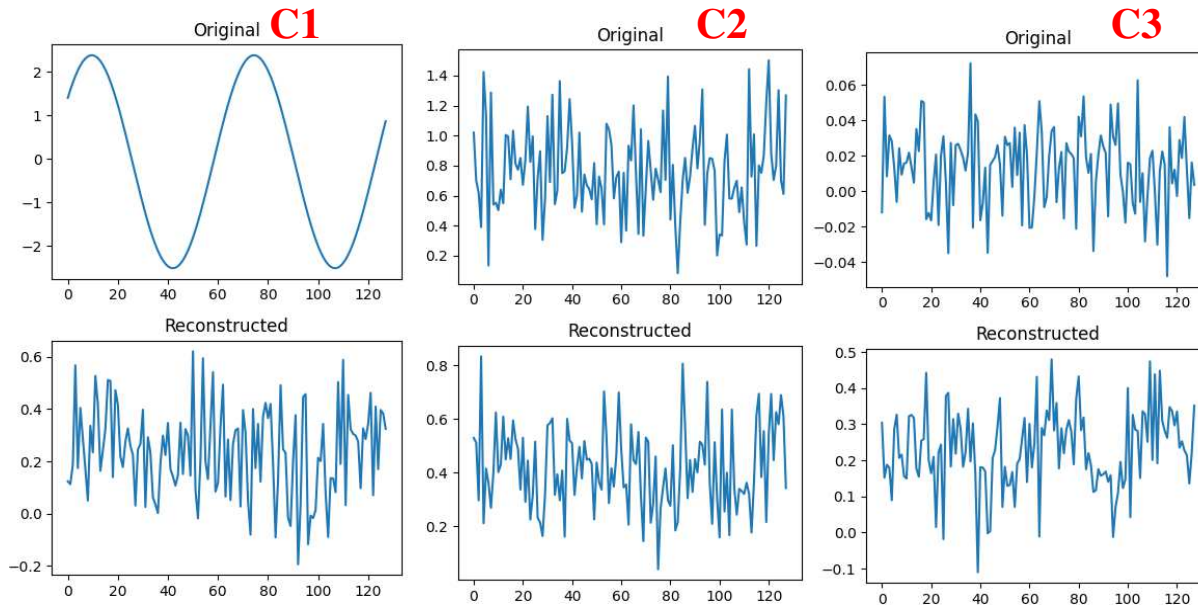
VAI_Slide_VAE_Signals_3Types_Beta_0_5.ipynb

β = balance / disentangle factor
 $\downarrow \beta$ makes Z space less norm-structured,
 but rebuilt $\uparrow$ (entangle)

With $\beta = 1$

VAI_Slide_VAE_Signals_3Types_Beta_1_0.ipynb
 VAI_Slide_VAE_Signals_3Types_Beta_0_5.ipynb
 VAI_Slide_VAE_Signals_3Types_Beta_0_05.ipynb

■ VAE ELBO = $\beta \times \text{KL loss}$ + rebuilt loss



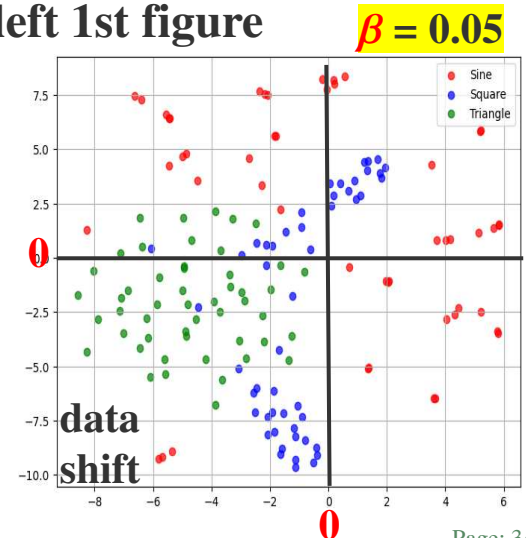
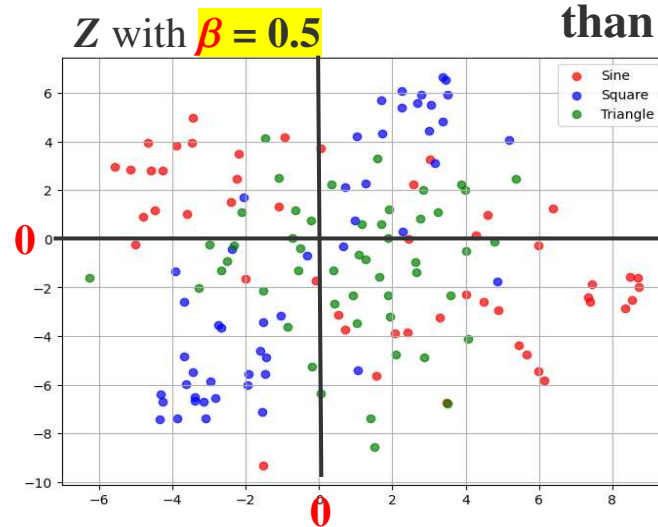
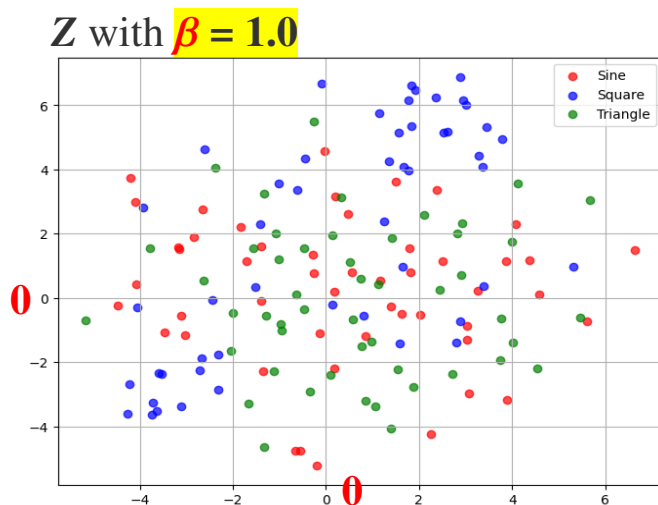
β = balance / disentangle factor

$\uparrow \beta$ makes Z space more

norm-structured,

but rebuilt $\downarrow$ (disentangle)

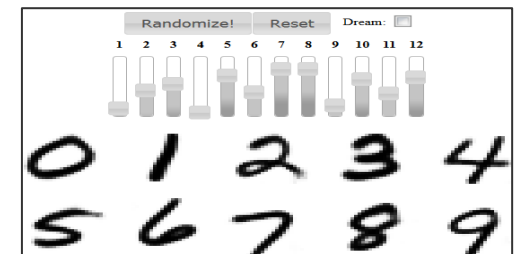
$\beta = 0.5 / 0.05$ pay attention
 in rebuilding data,
 better clusters on right
 than left 1st figure



Outline

- Variational AE (VAE) vs. AE
- VAE Architecture
- VAE Loss = KL Loss / Divergence + ELBO Loss
- Pseudo Code, Convolution / Dense, Sampling (how), Images / Numeric
- **Walking in Latent Space**
 - Better Controlled Generative AI

The Roots of Generative AI

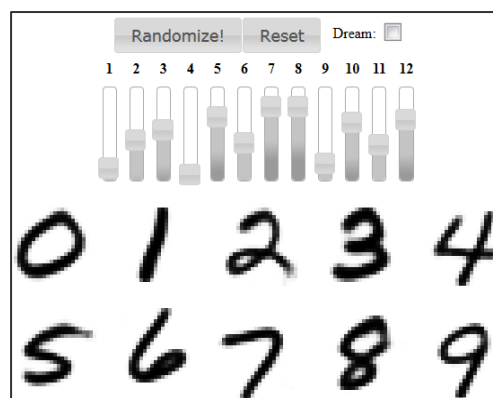
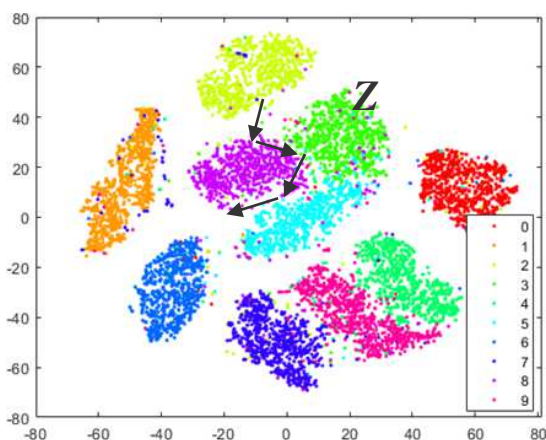


http://www.dpkimgma.com/sgvb_mnist_demo/demo.html

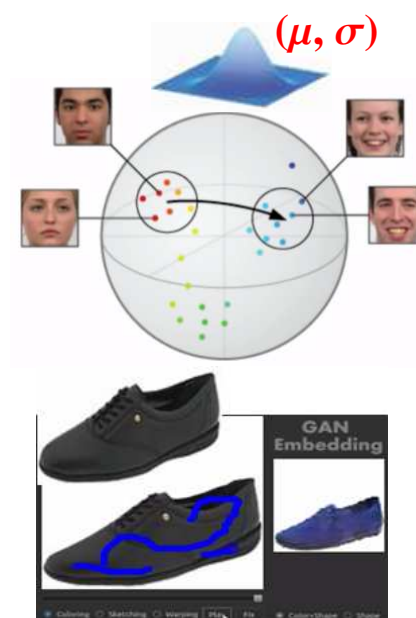


Walking in Z-Space using Generative AI

- How do we “*Walk in the Latent Space*”?? Applications??
 - Random???



http://www.dpkimgma.com/sgyb_mnist_demo/demo.html



<https://www.youtube.com/watch?v=9c4z6YsBGQ0>

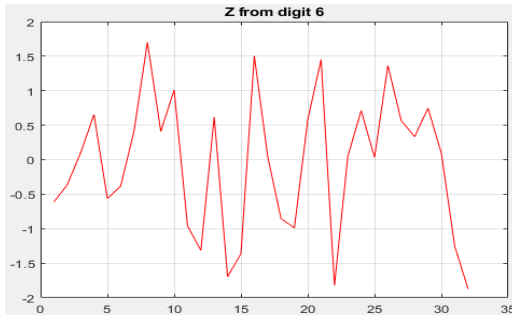


walking...

- How to control the walk in order to have good walking results????
 - By (1) changing Z directly , (2) changing σ or μ , or by (3) using two Z s

Method 1.

Just Change Z (randomly??), Changing Results Every Time, **WHY??**



% change certain **dim** in Z

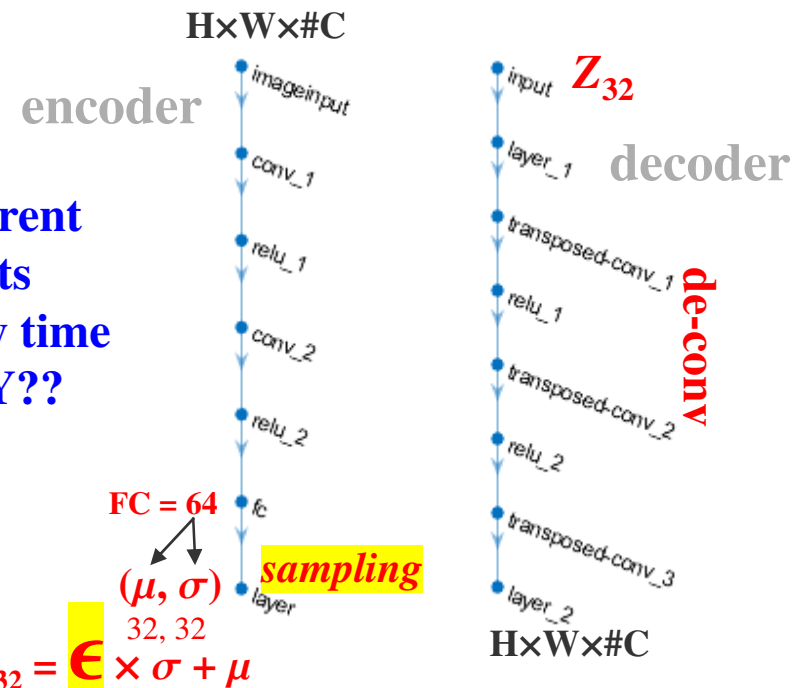
$Z1_Dim(1) = 0;$
 $Z1_Dim(4) = Z1_Dim(4) - 1;$
 $Z1_Dim(19) = Z1_Dim(19) - 1;$
 $Z1_Dim(23) = Z1_Dim(23) + 1.5;$

VAI_Slide_AE_Variational\
 VAI_Slide_VAE_Digits_CNN.m
 VAI_Slide_VAE_Digits_Manipulate.m
 VAI_Slide_VAE_Walking_Z_Override_FIT.ipynng

input *Enc* → Z → change Z → *Dec*

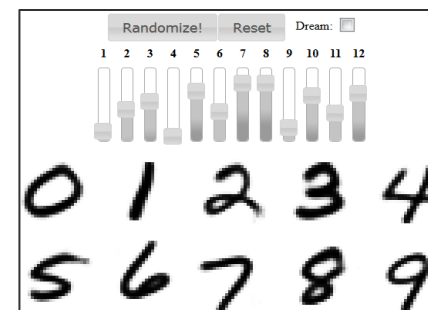


input *Enc* → Z → change Z → *Dec*



Method 2. Change Z via Amount p of Standard Deviation σ

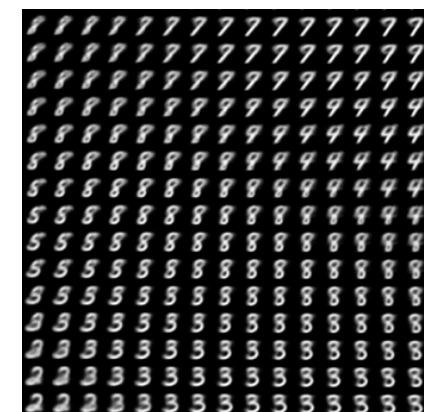
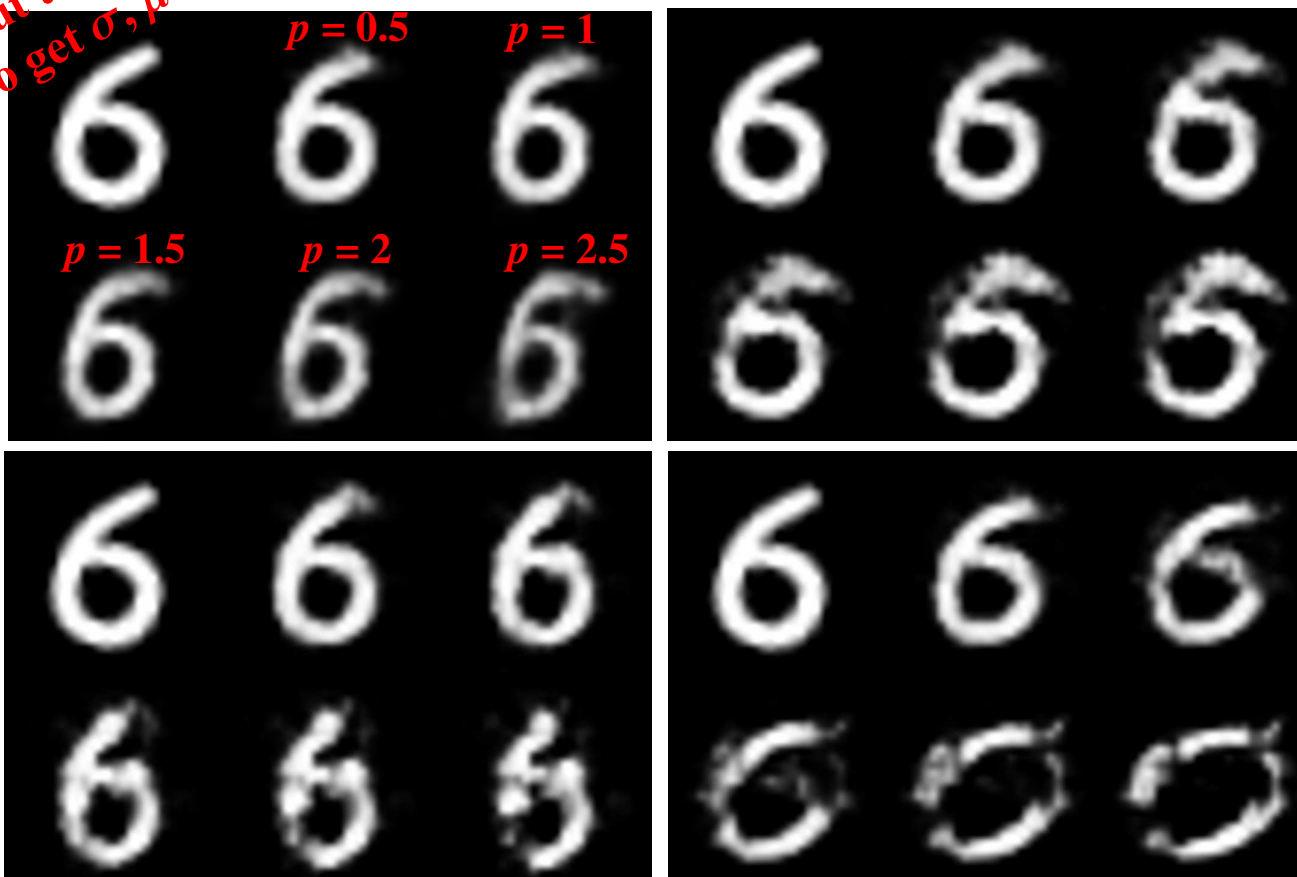
■ $Z = p \times \epsilon \times \sigma + \mu$ change Z via $p \times \epsilon \times \sigma$



http://www.dpkimgma.com/sgvb_mnist_demo/demo.html

change Z via $p \times \epsilon \times \sigma$

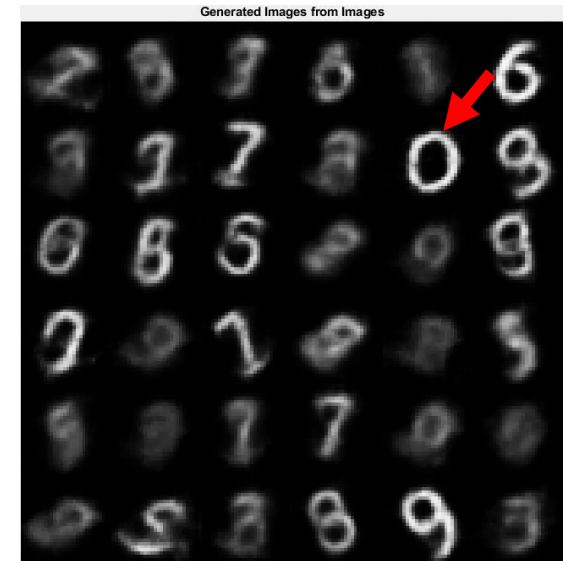
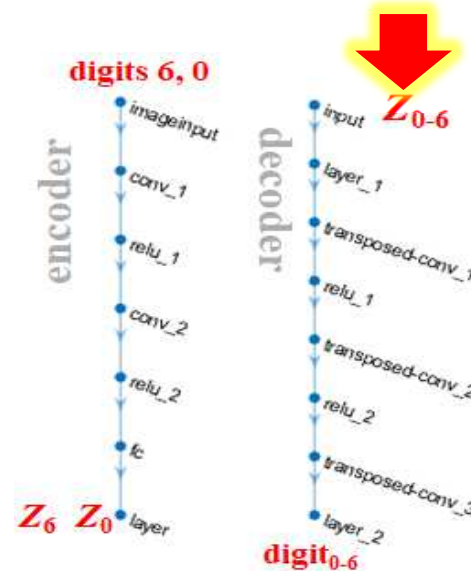
input to Enc
to get σ, μ



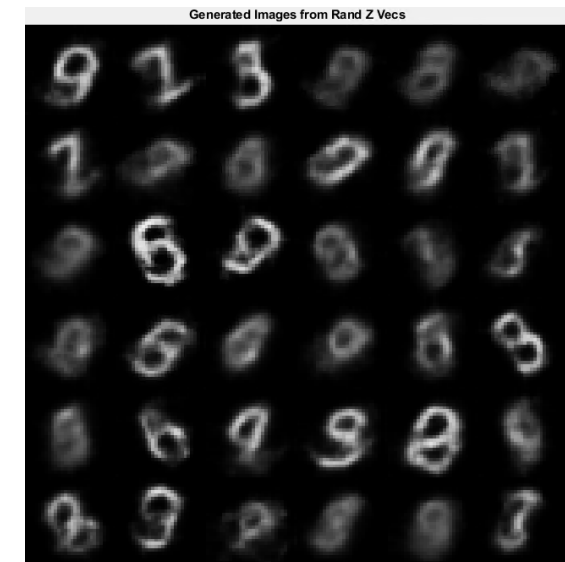
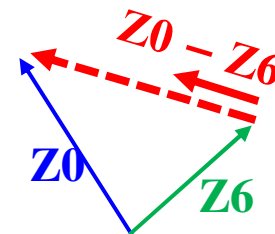
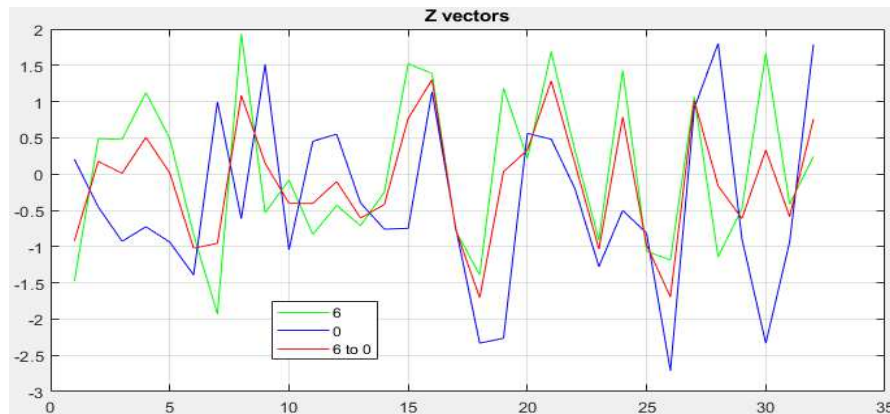
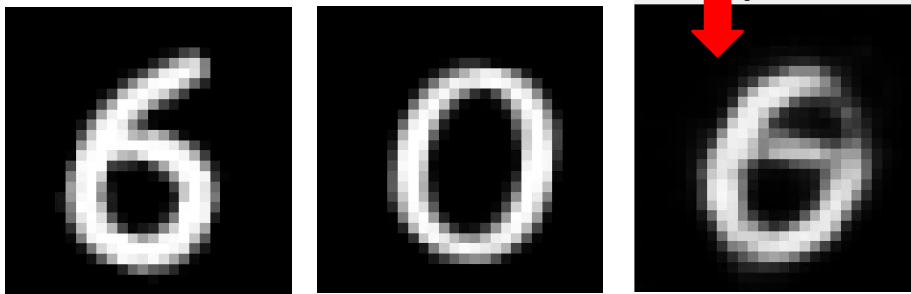
<https://keras.io/examples/generative/vae>

Method 3. Walking One Z Toward Another Z

- You can also walk toward the average of all Z_0 or the average of all $Z_{??} = \text{any digit}$



walking from digit 6 to digit 0



VAI_Slide_AE_Variational \

VAI_Slide_VAE_Digits_CNN.m

VAI_Slide_VAE_Digits_Manipulate.m

VAI_Slide_VAE_Walking_Z_Override_FIT.ipyn

Edit Keras VAE Example Program for *Walking* <https://keras.io/examples/generative/>

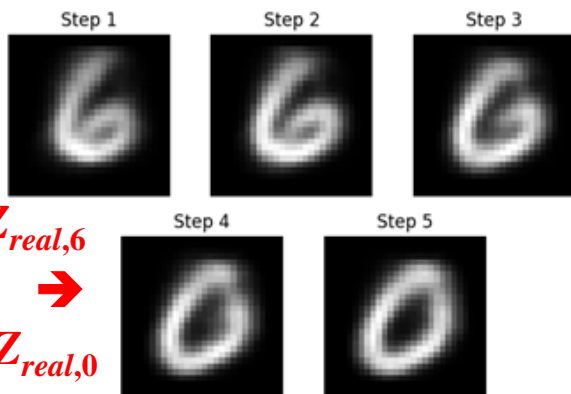
- 60,000 training images (28×28×1)
 - Remember to increase Z-dim from 2 to 30 for much better *walking*
 - **BUT**... the example uses “**fit**” for training...?! 😬
 - **Model.fit()** usually uses a default **train_step()**
 - But, this example **overrides** Keras’s default behavior in **train_step()** with **tf.GradientTape()** & **VAE losses**

Generative Deep Learning	
Image generation	
★ V3	Denoising Diffusion Implicit Models
★ V3	A walk through latent space with Stable Diffusion
V2	DreamBooth
V2	Denoising Diffusion Probabilistic Models
V2	Teach StableDiffusion new concepts via Textual Inversion
V2	Fine-tuning Stable Diffusion
★ V3	Variational AutoEncoder
V3	GAN overriding Model.train_step
V3	WGAN-GP overriding Model.train_step
V3	Conditional GAN
V2	CycleGAN
V2	Data-efficient GANs with Adaptive Discriminator Augmentation
V3	Deep Dream
V3	GauGAN for conditional image generation
V3	PixelCNN
V2	Face image generation with StyleGAN
V2	Vector-Quantized Variational Autoencoders
Style transfer	
V3	Neural style transfer
V2	Neural Style Transfer with AdaIN

VAI_Slide_VAE_Walking_Z_Override_FIT.ipynb

vae = VAE(encoder, decoder)

vae.fit(mnist_digits, epochs=30, batch_size=32)



$$Z(\alpha) = (1-\alpha) \cdot Z_{real,6} + \alpha \cdot Z_{real,0}$$

$\alpha = 0 \text{ to } 1$

class VAE(keras.Model):

```
def __init__(self, encoder, decoder, **kwargs):
    super().__init__(**kwargs)
    self.encoder = encoder
    self.decoder = decoder
    self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
    self.reconstruction_loss_tracker = keras.metrics.Mean()
```

def train_step(self, data): **override**
with **tf.GradientTape()** as tape:

```
z_mean, z_log_var, z = self.encoder(data) # reparameterization
reconstruction = self.decoder(z)
reconstruction_loss = ops.mean(
    ops.sum(ops.square(data - reconstruction), axis=(1, 2)) )
kl_loss = -0.5 * (1 + z_log_var - ops.square(z_mean) - ops.exp(z_log_var))
kl_loss = ops.mean(ops.sum(kl_loss, axis=1))
total_loss = reconstruction_loss + kl_loss
grads = tape.gradient(total_loss, self.trainable_weights)
self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
self.total_loss_tracker.update_state(total_loss)
self.reconstruction_loss_tracker.update_state(reconstruction_loss)
self.kl_loss_tracker.update_state(kl_loss)
return {
    "loss": self.total_loss_tracker.result(),
    "reconstruction_loss": self.reconstruction_loss_tracker.result(),
    "kl_loss": self.kl_loss_tracker.result(),
}
```

When to Use “fit(...)” and When to Use “Custom Train Loop”

Aspect	.fit() (Standard API)	Custom Loop (tf.GradientTape)
Logic	$X \rightarrow Y$, less flexibility w/ compile() then fit(). Handle forward+backward automatically	Use GradientTape + manual loss + manual optimizer step to handle forward + backward manually. Multiple Xs, Multiple Ys, max flexibility
Losses	Restricted to standard losses, (MSE, BCE/CE) or simple enough custom loss. Usage <code>compile(loss=.....)</code>	No limits. Your function calculate losses based on anything or many things together (layer weights, gradients, or multiple outputs).
Optimizers	Single optimizer in a single training step.	Use different optimizers in different steps to update different NNs
Layers	Standard layers.	Need @tf.function to transform user function into Tensor flow graph operation as layer of network.

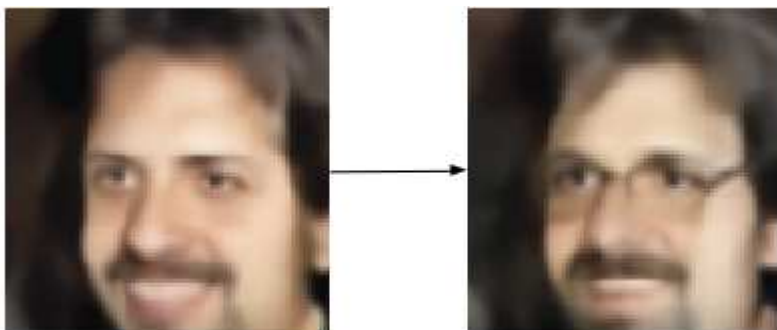
Customize training loop requires for →

- 1. Alternating Optimization** problems = multiple gradients to update different NNs
- 2. No Y** problems

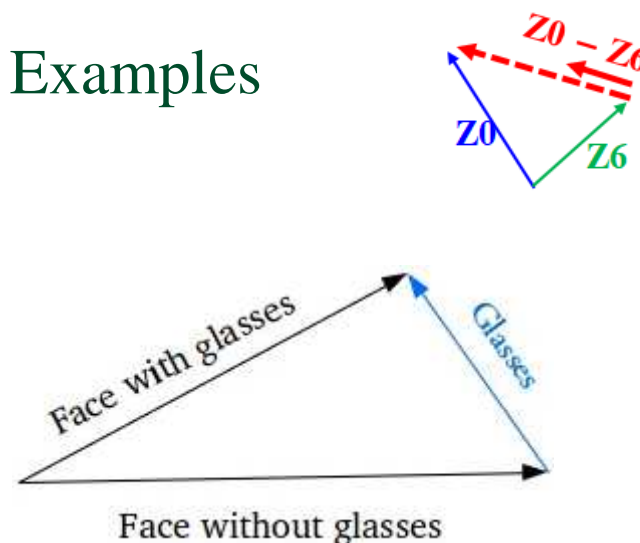
Walking in Latent Space, More Examples

$$\epsilon \times \sigma + \mu = Z$$

- Add Z w/ certain semantics in μ ... from
 - A specific image (i.e. male/female, smile/not), or
 - Extract images of one label & then average as μ



<https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>



<https://www.youtube.com/watch?v=9c4z6YsBGQ0>

for loop: $G_{\text{new}}(z) = G_{\text{old}}(z) + \epsilon$, walk 8 steps from one latent to different latent



→ walking...

American Sign Language

766, 25F
American sign language-
Image & video Analysis

Perry Abdulkadir
Bowen Lei
Jeremy Littlefield
Zhongke Wang

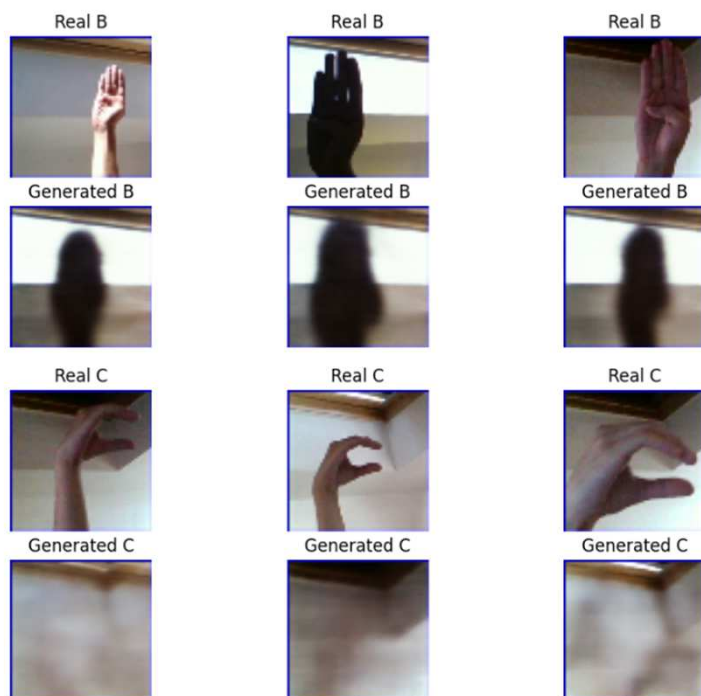
■ First version

- Standard **C/VAE** with BCE or MSE
- Shallow decoder (few Conv2DTranspose layers)

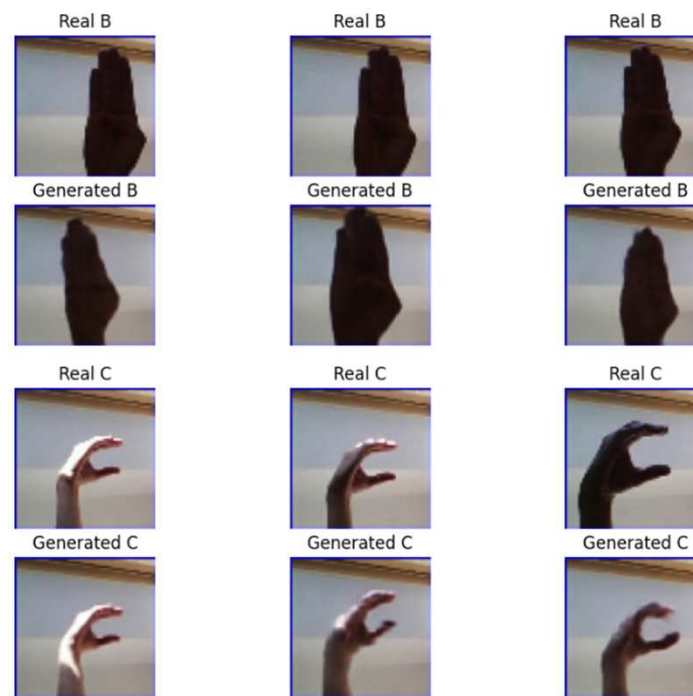
■ Fine-tuning (2nd) version

- Added more decoder layers→ produced sharper images
- Switched to $\text{MSE}(I, I') + \text{VGG perceptual loss}(Z, Z')$ → massive visual improvement
- Added batch normalization and learning rate tuning→ stabilized training

First Version

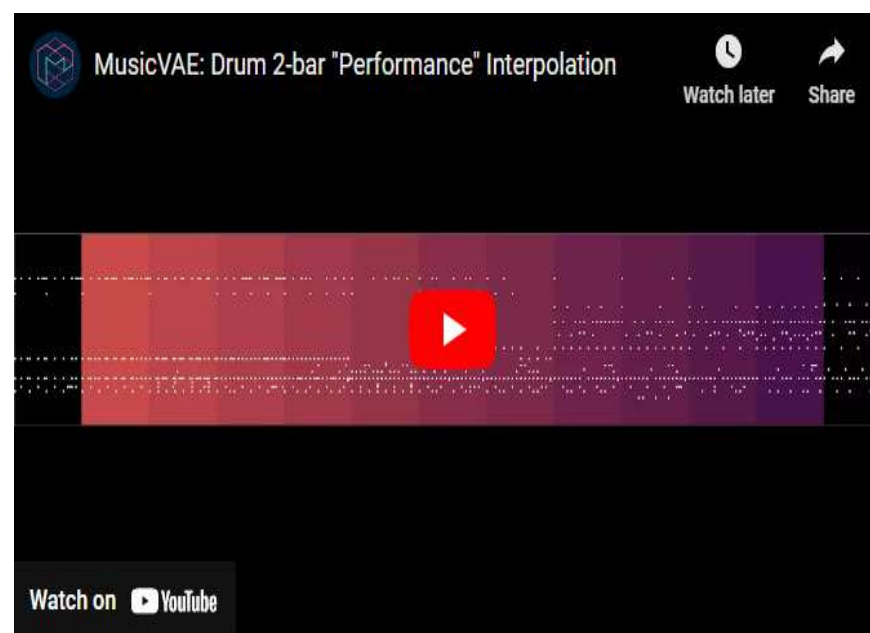
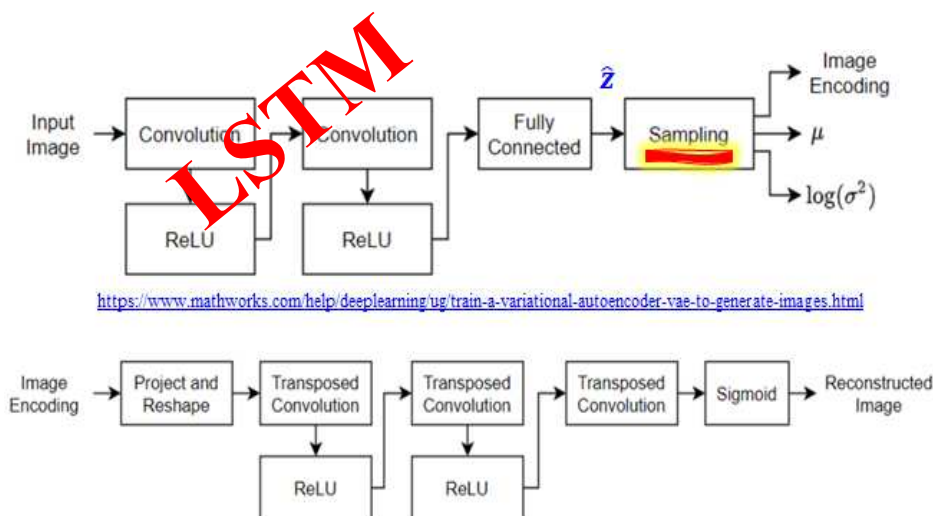
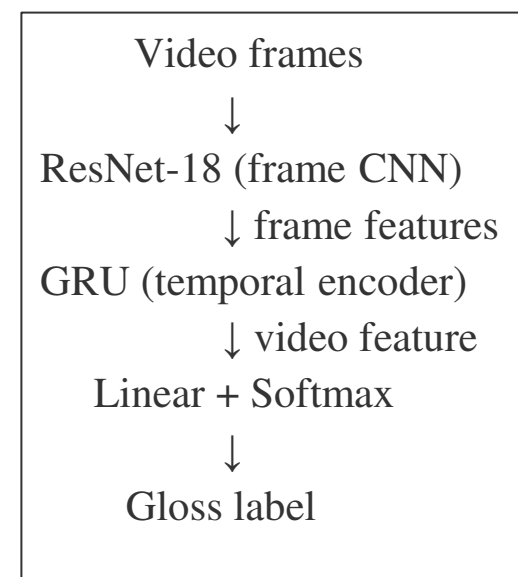


Final Version



VAE with LSTM for Sequential Data

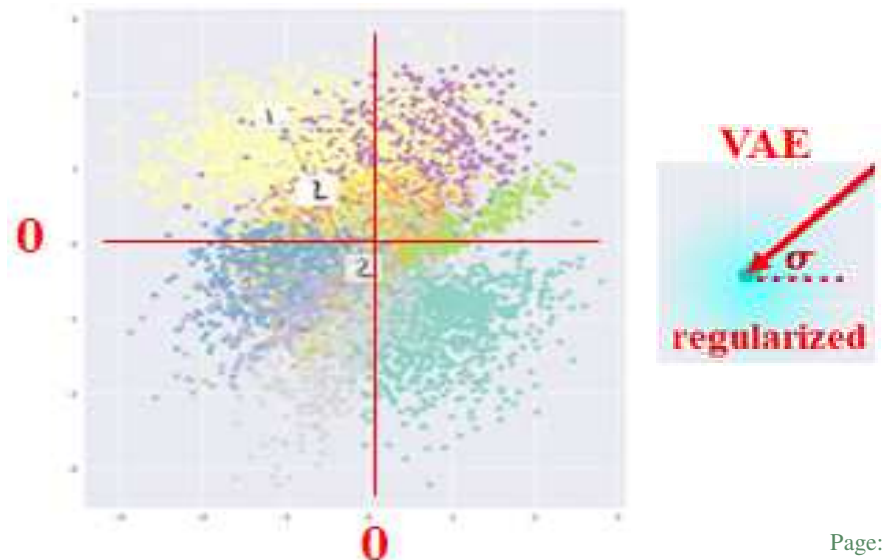
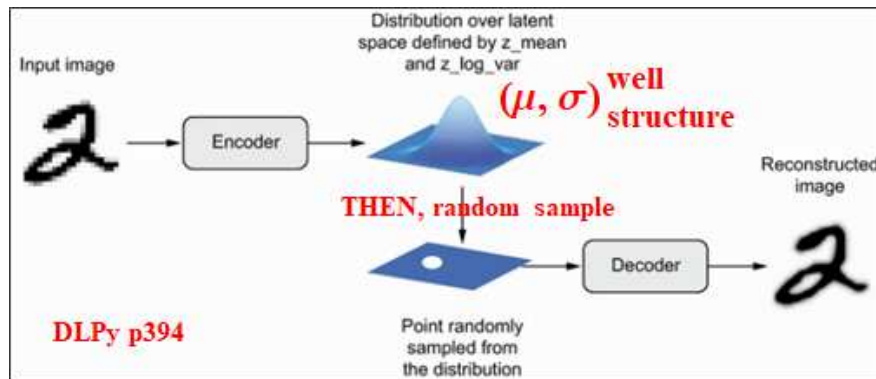
- VAE with LSTM for Sequential Data
- To produce synthetic text, or MIDI music
- i.e. Google Brain's Magenta's MusicVAE



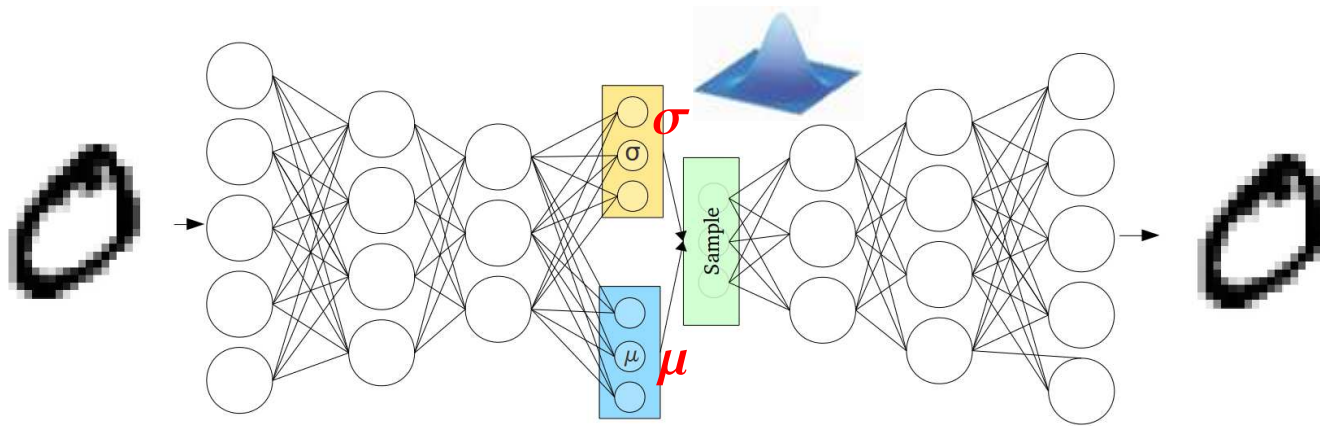
<https://www.youtube.com/watch?v=OxpP4IFp76I>

VAE Summary

- 2 separated but cooperative CNNs (GAN has 2 separated & competitive CNNs)
- **Encoder** generates Z anywhere inside distribution (μ, σ)
 - Describe inputs as distrib. (μ, σ) = continuous, easy for sampling & interpolation
 - Even with the same inputs, the actual encoding will vary due to **sampling**
 - Sampling layer actually produces $\log(\sigma^2)$ to make σ less scaled & more stable training
- **Decoder** learns a single point (μ, σ) in latent space is actually = a sample of a class
 - i.e. all nearby points refer to the **same class**
 - This allows decoder to decode not just 1 single specific output, but also **variations** too



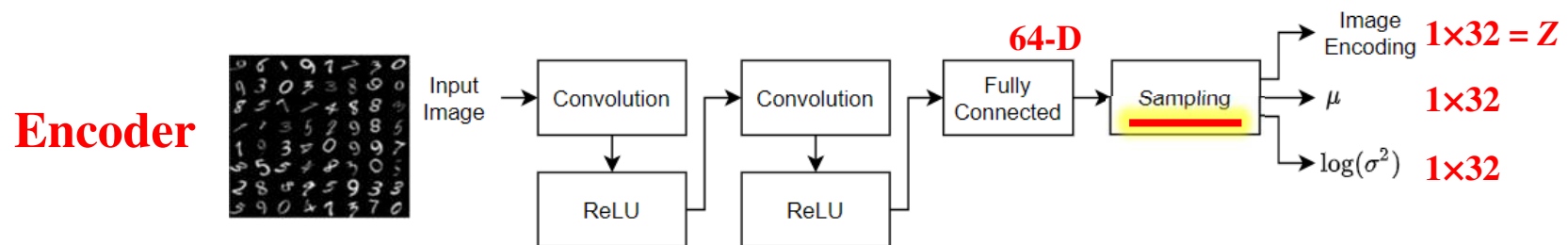
Appendix



<https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>

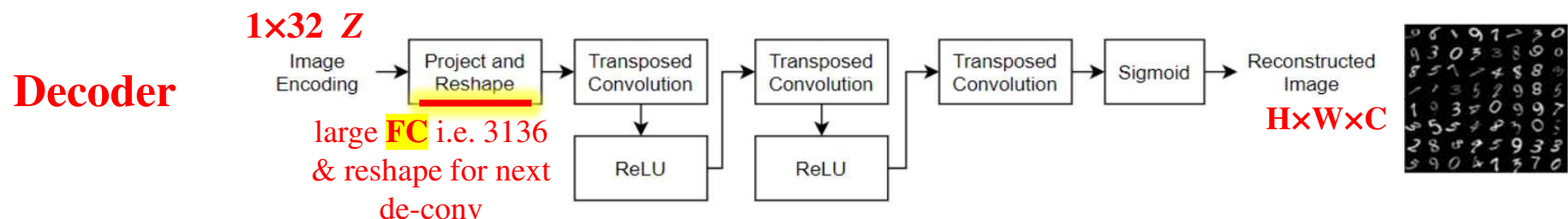
F.Y.I. Tricks in Training Encoder / Decoder

- For encoder training, do **not** normalize input images (CNN input layer **does** norm images)
 - VAE loss already learn compact latent representation, Norm provides no added value
 - Double norm leads to smaller range of values, masking subtle variations & diminishing small but important differences → harder for VAE to capture image info
 - **But** you need to norm / standardize **numeric** data (NN input layer does **NOT** norm data)



<https://www.mathworks.com/help/deeplearning/ug/train-a-variational-autoencoder-vae-to-generate-images.html>

- Decoder decodes from 1×32 latent **Z** vectors, rebuild $28 \times 28 \times 1$ image



Another Name of KL Divergence = **Relative Entropy** **F.Y.I.**

- **0 ≤ KL loss** = $-\sum P(x) \log\left(\frac{P(x)}{T(x)}\right) \leq \infty$
 - **KL loss** = $\sum_{i=1}^D [\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1]$ for **D**-var data
- **Entropy**($p_1 \dots p_n$) = $-p_1 \log_2 p_1 - p_2 \log_2 p_2 \dots - p_n \log_2 p_n$
- **Binary cross entropy** = $-Y_i \log(P_i) - (1 - Y_i) \log(1 - P_i)$

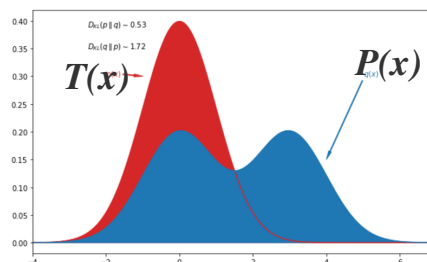
for each variable...

$$T = \left(\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}\right)$$

$$P = \left(\frac{1}{12}, \frac{1}{12}, \frac{1}{12}, \frac{1}{12}, \frac{1}{12}, \frac{1}{2}\right)$$

$$D_{KL}(T|P) = \sum_i P(i) \log\left(\frac{T(x)}{P(x)}\right)$$

$$D_{KL}(T|P) = \frac{1}{6} \log\left(\frac{1/6}{1/12}\right) + \frac{1}{6} \log\left(\frac{1/6}{1/12}\right) + \dots + \frac{1}{6} \log\left(\frac{1/6}{1/2}\right) = 0.3945$$



```
P = ones(1, 6) * (1/6);
Q = ones(1, 6) * (1/12); Q(end) = 1/2;
KL = 0;
for i=1:6
    KL = KL + P(i) * log(P(i) / Q(i));
end
% OR, just call following func
RE = relativeEntropy(P, Q)
```

def KL(a, b):

```
a = np.asarray(a, dtype=np.float)
b = np.asarray(b, dtype=np.float)
return np.sum(np.where(a != 0, a * np.log(a / b), 0))
```

```
values1 = [1.346112, 1.337432, 1.246655]
```

```
values2 = [1.033836, 1.082015, 1.117323]
```

```
print KL(values1, values2)
```

only accept CAT data

keras.losses.kl_divergence(y_true, y_pred)

Different Types of Cross Entropy

- **Sparse cross entropy**, WHY?? → efficient when you have a lot of categories
 - Too many categories consume huge amount of RAM when 1-hot encoded

- For each record i

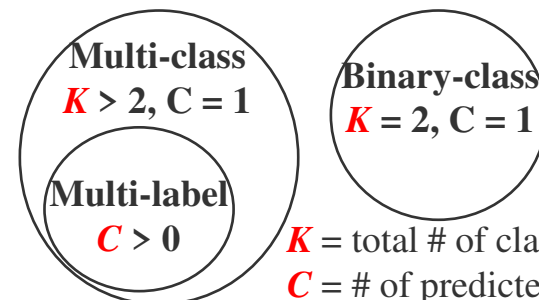
F.Y.I.

- **Binary cross entropy** $= -Y_i \log(P_i) - (1 - Y_i) \log(1 - P_i)$
 - Y_i be the true label of record i
- Multiclass **categorical cross entropy** $= \sum_{i=1}^C -Y_i \log(P_i) = -Y_i \log(P_i) = -\log(P_i)$
 - Y_i be each of all classes C that record i can possibly be predicted to
 - Formula can be simplified to $-\log(P_i)$ because of one-hot encoding
 - After simplification, P_i is the predicted probability of the true class
- Multiclass **sparse cross entropy** $= -Y_i \log(P_i) = -\log(P_i)$
 - Y_i be the true class label record i belongs, P_i is the predicted probability of the true class
 - Formula can be simplified because each class can only belong to one true class

- **Multilabel** cross entropy

$$= \frac{1}{L} \sum_{i=1}^L -Y_i \log(P_i)$$

- $L = \underline{\text{multiple}}$ classes i can belong to



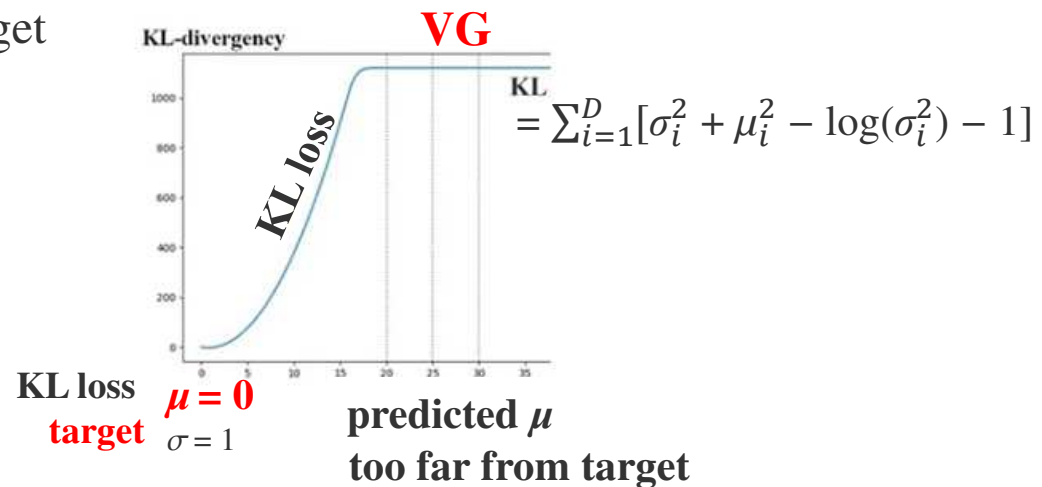
K = total # of classes
 C = # of predicted classes for each record

Training NN with KL Loss May Have Potential Vanish Gradient

- **KL divergence** is commonly used to compare 2 distributions



- But, for training VAE, the KL divergency may have the vanishing gradient problem if predicted posterior distribution becomes too far from target (*normal*) distribution
 - i.e. prediction too far from target



<https://jonathan-hui.medium.com/gan-wasserstein-gan-wgan-gp-6a1a2aa1b490>

- Solution?? **Wasserstein Distance**
- But **KL** is **much more** computational efficient **in training**

VAE Implementation

VAI_Slide_VAE_CNN_Digits.ipynb

- **E** encodes input & gen latent vectors (μ, σ) , **D** samples (μ, σ) & rebuild output $\approx$ input
- **HOW** to sample from (μ, σ) as $Z = \epsilon \times \sigma + \mu$ for input to decoder??
 - **Lambda** = lambda layer, a simple way to wrap any custom math operations as a layer
 - Programming, an independent layer

```
E_Mdl, ConvShape = create_E(in_shape, Z_dim)
D_Mdl = create_D(Z_dim, in_shape, conv_shape)
vae, z_extractor = connect_vae(E_Mdl, D_Mdl, in_shape, Z_dim)
```

```
encoder_inputs = keras.Input(shape=input_shape,)
x = encoder_inputs
x = layers.Conv2D(32, 3, activation="relu", strides=2, padding="same")(x)
x = layers.Conv2D(64, 3, activation="relu", strides=2, padding="same")(x)
```

create_E()

Store the shape before flattening for the *****decoder*****

```
conv_shape = tf.keras.backend.int_shape(x)
```

Flatten to connect to the latent layer

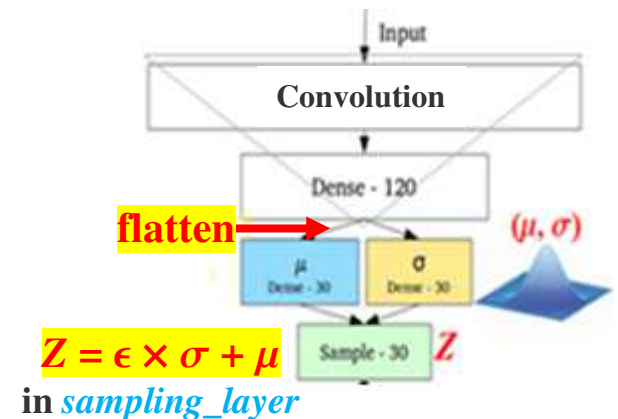
```
x = layers.Flatten()(x)
x = layers.Dense(16, activation="relu")(x)
```

output latent distribution **Sampling Z??**

```
z_mean = layers.Dense(Z_dim)(x)
z_log_var = layers.Dense(Z_dim)(x)
```



```
return keras.Model(encoder_inputs,
                    [z_mean, z_log_var], conv_shape)
```



```
# connect_vae(E_Mdl, D_Mdl, in_shape, Z_dim)
vae_inputs = keras.Input(shape=input_shape, name="vae_input")

# 1. Forward pass through the encoder
z_mean, z_log_var = encoder_model(vae_inputs)

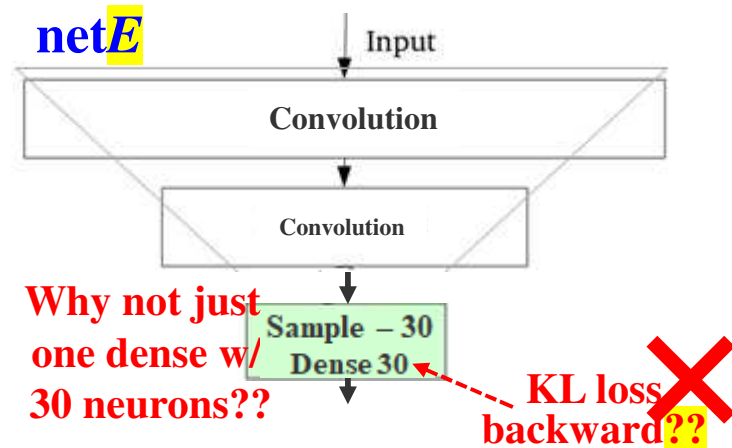
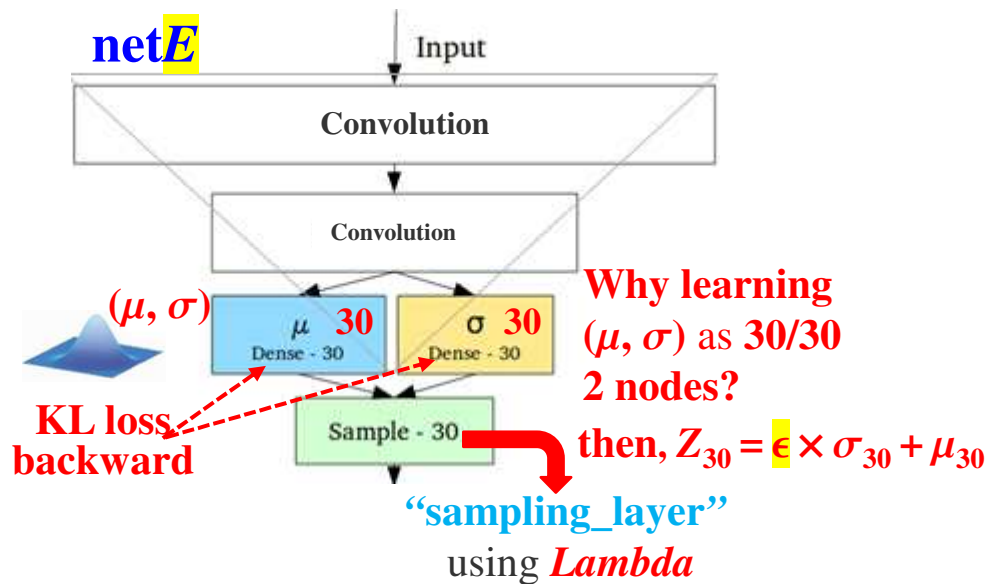
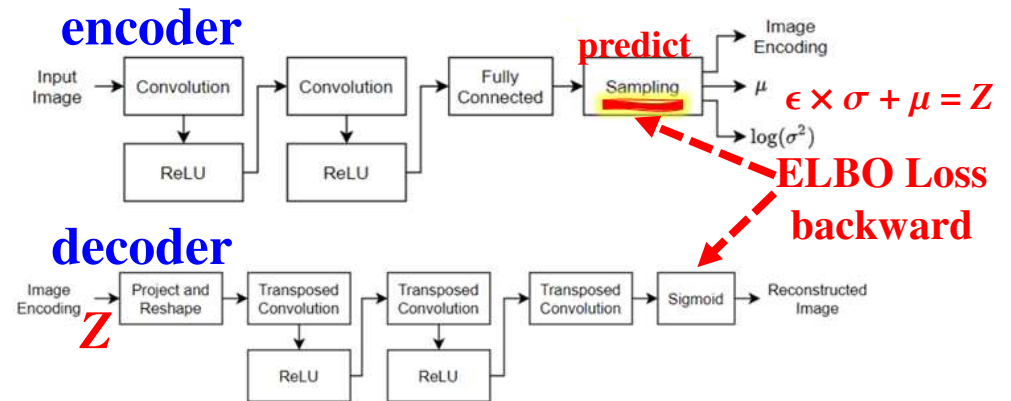
# 2. ***** Reparameterization step for "Sampling Layer"
z = layers.Lambda(WHY/What lambda??)
    (sampling_layer, output_shape=(Z_dim,))([z_mean, z_log_var])

# 3. Forward pass through the decoder
reconstruction = decoder_model(z)
```

$Z = \epsilon \times \sigma + \mu$

VAE Backward Propagation

VAI_Slide_VAE_CNN_Digits.ipynb



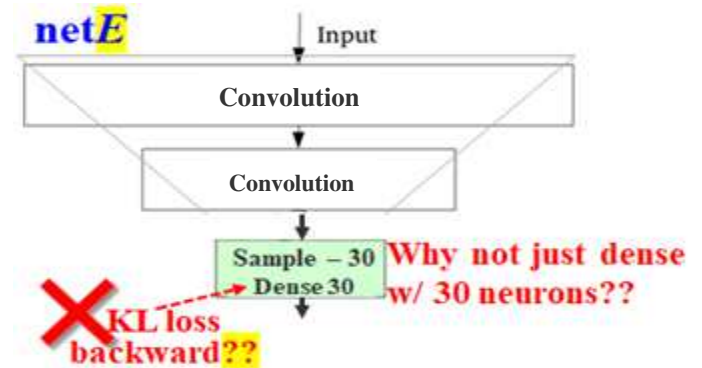
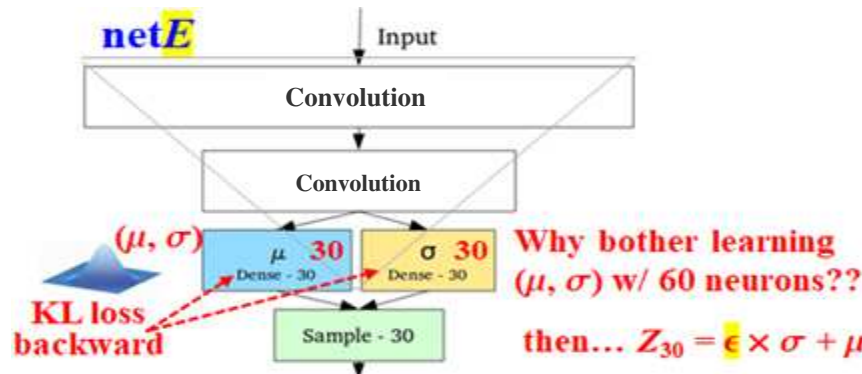
■ How can gradient back-propagate possible over *non-differentiable* sampling??

• **Reparametrization trick** $\Rightarrow Z_{30} = \epsilon \times \sigma + \mu$

else non-differentiable
from concept of $Z \sim N(\mu, \sigma^2)$

• Implement a customized **"sampling_layer"** using **Lambda**

Implementation of *Reparameterization*



- How can gradient back-propagate possible over *non-differentiable* sampling layer ??

- **Reparameterization trick** to get $Z_{30} = \epsilon \times \sigma + \mu$
- Implement “**sampling_layer**” using **Lambda**

2. ***** Reparameterization step for “Sampling Layer”
 $z = \text{layers.Lambda}$ WHY/What **lambda**??
 (**sampling_layer**, output_shape=(**Z_dim**,))([**z_mean**, **z_log_var**])

reparameterization in a custom layer to be called by **Lambda**

def **sampling_layer**(inputs):

z_mean, **z_log_var** = inputs # μ, σ^2

batch = tf.shape(z_mean)[0]

dim = tf.shape(z_mean)[1]

Sample epsilon from standard normal distrib.

epsilon = keras.backend.random_normal(shape=(batch, dim))

Calculate standard deviation: $\exp(0.5 * z_log_var)$

sigma = tf.exp(0.5 * **z_log_var**)

Final latent vector z

return **z_mean** + **sigma** * **epsilon** → $Z = \mu + \sigma \times \epsilon$

VAI_Slide_VAE_CNN_Digits.ipynb

σ = standard deviation, σ^2 = variance
 $\log(\sigma^2)$ = log variance

$\sigma = \sqrt{\sigma^2} = \sqrt{e^{\log(\sigma^2)}}$ since VAR learn $\log(\sigma^2)$
 $= (e^{\log(\sigma^2)})^{1/2} = e^{0.5 \times \log(\sigma^2)}$